

Reprodução do paper Alsaedi 2020 e avaliação da família WiSARD para detecção de anomalia em IoT (TON_IoT)

Relatório técnico do Trabalho 1

Turma 2026.1 – Redes Neurais Sem Peso
Programa de Engenharia de Sistemas e Computação (PESC)
COPPE – Universidade Federal do Rio de Janeiro

Maio de 2026

Resumo

Este relatório consolida o Trabalho 1 da disciplina de Redes Neurais Sem Peso (WNN), em que avaliamos cinco modelos da família WiSARD (WiSARD clássica, ClusWiSARD, BloomWiSARD, BTHOWeN e ULEEN) na tarefa de detecção de anomalia em telemetria IoT, usando o dataset TON_IoT (UNSW Canberra Cyber). Como referência metodológica e numérica usamos o paper *Alsaedi et al. 2020* (IEEE Access), reproduzindo fielmente os oito *baselines* reportados (LR, LDA, kNN, RF, CART, NB, SVM e LSTM) sob o protocolo descrito na Seção VI-A do paper (*split* 80/20, validação cruzada $k = 4$ no *train*, *F1 weighted*). Durante a reprodução, identificamos três problemas estruturais nos CSVs públicos do TON_IoT que não são discutidos no paper original: (i) *leak* categórico determinístico em `Fridge.temp_condition` e `Garage_Door.sphone_signal` (Cramér's $V = 1,000$); (ii) *leak* temporal universal — o campo `date` prevê classe perfeitamente em todas as sete bases; (iii) divergência de tamanho entre os CSVs públicos (`Train_Test_IoT_*.csv`) e o dataset original que produziu as Tabelas 10–13 do paper (classe `normal` sub-amostrada para 43% do volume original). Sob o protocolo *paper-faithful* adotado, e sob tetos de capacidade *unificados* entre os modelos, a família WiSARD fica a poucos pontos percentuais dos melhores *baselines* em bases sem *leak* forte: WiSARD e ClusWiSARD lideram o grupo praticamente empatadas ($F1 \approx 0,82$), a cerca de um ponto percentual de Random Forest e à frente de kNN, SVM e LSTM. No *trade-off* $F1 \times$ custo há um *trade-off* de memória dentro da própria família: as variantes de filtro (BloomWiSARD, BTHOWeN, ULEEN) têm *footprint* fixo de 0,75–2 *KB*, cerca de três ordens de magnitude abaixo dos modelos de árvore, enquanto WiSARD e ClusWiSARD recuperam o $F1$ mais alto a ≈ 2 *KB* medianos (dependentes dos dados); o BTHOWeN treina cerca de $705\times$ mais rápido que o LSTM.

Palavras-chave: WiSARD, WNN, TON_IoT, detecção de anomalia, IoT, BTHOWeN, ULEEN.

1 Introdução

A proliferação de dispositivos de *Internet of Things* (IoT) e *Industrial IoT* (IIoT) ampliou drasticamente a superfície de ataque das redes corporativas e residenciais. Sistemas de detecção de intrusão (*Intrusion Detection Systems*, IDS) baseados em aprendizado de máquina vêm sendo a abordagem dominante para esse cenário, especialmente quando aplicados sobre telemetria coletada diretamente nos sensores — temperatura, posição, sinal de porta, leituras Modbus. Dois requisitos competem no *deployment* em *edge*: (i) eficácia preditiva alta o suficiente para que falsos positivos não inundem o operador, e (ii) custo computacional baixo o bastante para caber em hardware embarcado de poucos μW .

Esse trabalho avalia, sob esse duplo critério, a **família WiSARD** de redes neurais sem peso (*Weightless Neural Networks*, WNN) [2, 1] contra os algoritmos clássicos de aprendizado de máquina sobre o dataset **TON_IoT** [4]. O TON_IoT é a referência mais recente da comunidade

para IDS em IoT/IIoT, e o paper original de Alsaedi et al. (IEEE Access, 2020) estabelece uma linha de base extensa com oito modelos clássicos — Logistic Regression (LR), Linear Discriminant Analysis (LDA), k -Nearest Neighbors (kNN) [9], Random Forest (RF) [7], CART, Naïve Bayes (NB), Support Vector Machine (SVM) e Long Short-Term Memory (LSTM) [11] — avaliados em sete bases de telemetria e em uma base consolidada.

Pergunta de pesquisa. A família WiSARD, projetada explicitamente para baixa demanda computacional, consegue se aproximar da eficácia preditiva dos *baselines* clássicos no TON_IoT quando avaliada *sob o mesmo protocolo experimental do paper de referência*?

Contribuições. Este relatório consolida os seguintes resultados:

1. **Reprodução *paper-faithful* dos oito *baselines*** de Alsaedi 2020 sob o protocolo descrito em sua Seção VI-A (*split* 80/20, $k = 4$ *StratifiedKfold* no *train*, métrica *F1 weighted* adotada após comparação empírica). Os números obtidos batem com o paper dentro de $|\Delta| < 0,15$ em 91% dos pares (modelo, base).
2. **Avaliação de cinco modelos da família WiSARD** (WiSARD [2], ClusWiSARD [8], BloomWiSARD [16], BTHOWeN [20], ULEEN [19]) sob o *mesmo* protocolo — comparação direta sob protocolo idêntico aos *baselines* do paper.
3. **Auditoria de qualidade do dataset TON_IoT**, em que identificamos três problemas estruturais não documentados no paper: *leak* categórico determinístico em duas bases, *leak* temporal universal e divergência de tamanho entre os CSVs públicos e a versão usada nas Tabelas 10–13 do paper.
4. **Extensões na biblioteca `wisardpkg`**: um conjunto curado de termômetros não supervisionados (*Distributive*, *Gaussian*, *Exponential*, *Logarithmic* e a alocação *Non-uniform*) usado na análise de sensibilidade, e uma métrica unificada de estado implantado (`deployedSizeBytes`) que coloca todas as variantes sem peso no mesmo eixo de memória.

Estrutura do documento. A Seção 2 apresenta o paper de referência e os modelos da família WiSARD. A Seção 3 reporta a auditoria de qualidade do TON_IoT — *leaks* categóricos e temporais identificados, e a divergência de tamanho entre os CSVs públicos e o paper. A Seção 4 formaliza o protocolo *paper-faithful* usado para todas as comparações. A Seção 5 reporta a reprodução dos oito *baselines*, validando o pipeline com sanity checks derivados das Tabelas 10–13 do paper. A Seção 6 apresenta o *sweep* de hiperparâmetros dos cinco modelos da família WiSARD. A Seção 7 consolida as três visões lado-a-lado: paper, nossa reprodução e WiSARD-family. A Seção 8 discute limitações honestas da reprodução, e a Seção 9 encerra com as conclusões.

2 Trabalho relacionado

2.1 O dataset TON_IoT e o paper de Alsaedi 2020

O TON_IoT [4, 13] foi construído pela equipe da UNSW Canberra Cyber e cobre três camadas distintas do tráfego de uma rede IoT: tráfego de rede, eventos de sistema operacional e **telemetria de sensores**. Este relatório foca exclusivamente na camada de telemetria, que contém sete bases: **Fridge**, **Garage_Door**, **GPS_Tracker**, **Modbus**, **Motion_Light**, **Thermostat** e **Weather**, totalizando cerca de 260 mil linhas, mais uma base **combined** (concatenação das sete, usada nas Tabelas 12–13 do paper). Cada amostra é rotulada como **normal** ou como um dos sete tipos de ataque (**backdoor**, **ddos**, **injection**, **password**, **ransomware**, **scanning**, **xss**; nem toda base contém todos os tipos).

O paper de Alsaedi et al. avalia oito *baselines* clássicos nessa telemetria sob três tarefas: (i) classificação binária *per-device* (Tabelas 10–11 do paper), (ii) classificação binária sobre o

combined (Tabela 12), e (iii) classificação multi-classe sobre o *combined* (Tabela 13). O protocolo, descrito na Seção VI-A, é *split* 80%/20% com validação cruzada $k = 4$ no *train*, e a métrica reportada é F-score, embora a chamada exata da função (**binary**, **macro**, ou **weighted**) não seja explicitada. Tratamos essa ambiguidade na Seção 8.

2.2 Vazamento de dados e qualidade do TON_IoT

A comunidade já documentou vazamento de dados (*data leakage* — quando uma *feature* carrega informação do rótulo que não estaria disponível em produção, inflando artificialmente a acurácia) no TON_IoT, porém sempre no **dataset de rede** e por *features* identificadoras. Shaikhanova et al. [17] mostram que o resultado de 99,97% do estudo original incluía endereços IP e números de porta como *features*, atributos que “introduzem vazamento e superestimam o desempenho real”, e propõem descartar oito colunas textuais de alta cardinalidade. Dharini et al. [10] fazem a mesma crítica: os ataques foram lançados de faixas de IP e portas pré-definidas, fazendo esses campos agirem como identificadores de ataque em vez de indicadores de comportamento. Raskovalov et al. [15] vão além e publicam uma versão corrigida (ToN-IoT-R), filtrando fluxos rotulados como ataque que continham tráfego normal com o roteador e o *resolver* DNS. Todas essas correções operam sobre colunas (**ssl_subject**, **http_uri**, IP, porta) que *inexistem* na telemetria de sensores — portanto não alcançam o vazamento que identificamos.

Na camada de **telemetria**, ao contrário, a acurácia quase-perfeita é tratada como virtude, e não como sintoma. Sharrab et al. [18] dedicam um artigo ao **Garage_Door** e reportam 100% de acurácia para múltiplos classificadores, atribuindo o resultado ao “poder preditivo das *features* mais correlacionadas” sem levantar qualquer alarme; Alotaibi e Ilyas [3] combinam seis dispositivos de telemetria e reportam 98,64% sem uma seção de limitações sobre a acurácia. Benaddi et al. [5] reportam macro-F1 acima de 0,986 no TON_IoT com um IDS comprimido (poda de *features* guiada por SHAP e distilação de conhecimento), tratando a acurácia quase-perfeita como resultado a otimizar e não como sintoma a investigar, sem qualquer análise de vazamento. A crítica de heterogeneidade de Booij et al. [6], a mais citada sobre o dataset, trata de generalização entre versões, não de vazamento.

Nenhum desses trabalhos inspeciona os CSVs de telemetria no nível da *feature* categórica, e nenhum conecta a determinismo da associação à sub-amostragem da classe **normal** nos arquivos públicos. É exatamente esse o foco da nossa auditoria (Seção 3): diagnosticamos um vazamento categórico determinístico (Cramér’s $V = 1,000$) em **Fridge** e **Garage_Door** e o rastreamos até a sub-amostragem $35k \rightarrow 15k$ da classe **normal**, um mecanismo distinto do vazamento de identificadores da camada de rede e, até onde alcançamos na literatura, ainda não reportado na telemetria.

2.3 Família WiSARD de redes neurais sem peso

Redes neurais **sem peso** (WNNs) substituem produtos peso-entrada típicos das redes convencionais por consultas $\mathcal{O}(1)$ em *RAMs* indexadas por padrões binários de entrada. A versão seminal — **WiSARD** [2] — é o *discriminador* mais simples possível: um conjunto de *RAMs* $r_1, \dots, r_N$, cada uma endereçada por uma tupla de n bits (*tuple address*) sorteada (mapeamento pseudo-aleatório) sobre a representação binária da entrada. Para cada classe, treinar significa marcar com 1 as posições endereçadas pelas amostras dessa classe; classificar significa contar quantas *RAMs* respondem 1 para a entrada candidata e escolher a classe com maior contagem.

A simplicidade tem custo: a representação binária da entrada precisa preservar a estrutura métrica das *features*. A solução clássica é o **termômetro**, que codifica um valor real $x \in [a, b]$ em T bits, com os primeiros $\lceil T \cdot (x - a) / (b - a) \rceil$ ligados. Variantes ajustadas aos dados (distributivo, gaussiano, exponencial, logarítmico) adaptam a discretização à distribuição empírica da *feature*; avaliamos seis dessas variantes na Seção 4.

Os cinco modelos avaliados estendem o esqueleto WiSARD em direções ortogonais:

WiSARD [2]. O modelo base. Hiperparâmetros principais: tamanho do endereço (n), termômetro e número de *bits* por *feature*, `ignore_zero` (descartar endereços nulos).

ClusWiSARD [8]. Particiona cada classe em *sub-discriminadores* via critério de similaridade (`min_score`, `threshold`, `limit`). Útil quando a classe tem sub-modos estatisticamente distintos.

BloomWiSARD [16]. Substitui a RAM tradicional por um *Bloom filter*, o que reduz drasticamente o consumo de memória ao custo de falsos positivos controlados.

BTHOWeN [20]. Adota *hashed lookup tables* de tamanho fixo (`filter_entries`) com k funções de hash (`num_hashes`). Projetado para implementação em FPGA com latência sub-microsegundo.

ULEEN [19]. Adiciona uma fase adicional de *learning rate* sobre as RAMs e um esquema de *bleaching* treinável, alcançando F1 superior ao BTHOWeN em datasets de visão ao custo de mais iterações.

Os experimentos usam a biblioteca `wisardpkg` [12] estendida com o conjunto de termômetros ajustados descrito na Seção 4. Os modelos *baseline* usam `scikit-learn` [14] para LR/LDA/kNN/RF/CART/NB/SVM e PyTorch para o LSTM, sempre com semente fixa (`random_state=42`) em todos os nossos experimentos; o paper não publica a semente que usou (ver §8).

3 Auditoria de qualidade do dataset TON_IoT

Antes de treinar qualquer modelo, fizemos uma auditoria da qualidade estatística dos sete CSVs de telemetria disponibilizados pela UNSW (`Train_Test_IoT_*.csv`). A motivação é simples: o paper Alsaedi 2020 reporta F1 igual a 1,00 em várias bases (notavelmente `Garage_Door` em todos os oito modelos), e qualquer reprodução que não interrogue *por que* isso acontece corre o risco de incorporar artifacts não-conceituais e declarar vitória indevida. Encontramos três problemas estruturais: dois tipos de *leak* e uma divergência crítica de tamanho entre os CSVs públicos e o que o paper de fato mediu.

3.1 Inventário de features e Cramér’s V

A Figura 1 mostra, para cada base, o maior Cramér’s V entre as *features* contra o rótulo binário (`normal` vs qualquer ataque), calculado sobre os dados crus. Valores próximos de 1 indicam uma *feature* deterministicamente associada ao rótulo — um sinal forte de *leak*.

3.2 Leak categórico em duas bases

Duas *features* categóricas mostraram Cramér’s V exatamente 1,000 contra o rótulo (Tabela 1):

Tabela 1: Leaks categóricos identificados nos CSVs públicos.

Base	Feature	Causa	V
Fridge	<code>temp_condition</code>	6 variantes <i>whitespace</i> de <code>'high'/'low'</code>	1,000
Garage_Door	<code>sphone_signal</code>	<code>'0'/'1'</code> (atk), <code>'false'/'true'</code> (normal)	1,000

Em `Fridge.temp_condition`, as seis variantes (`'high'`, `'high '`, `'high '` e as análogas para `'low'`) diferem apenas por caracteres de *whitespace* no final da string. Para um humano elas são a mesma palavra; para um `LabelEncoder` ingênuo, são seis classes distintas, e o modelo aprende a separar `normal/attack` via essa diferença ortográfica em vez de via o conceito que a *feature* representa: a variante *sem* espaço aparece 100% em `attack`, as variantes *com* espaço 100% em `normal`. Análogo em `Garage_Door`, em que o encoding mistura strings booleanas (`'false'/'true'`, sempre `normal`) com strings numéricas (`'0'/'1'`, sempre `attack`) de

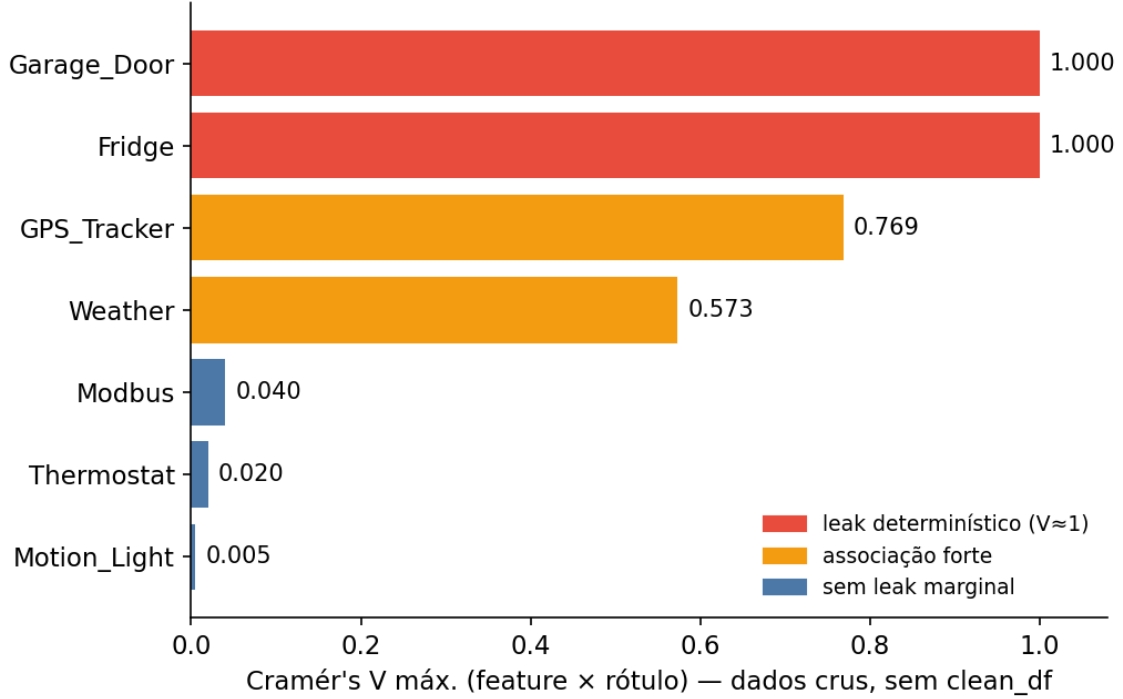


Figura 1: Cramér's V máximo por base (a *feature* mais associada ao rótulo), sobre os dados crus. `temp_condition` (Fridge) e `sphone_signal` (Garage_Door) atingem $V = 1,000$, uma dependência determinística (leak). `GPS_Tracker` e `Weather` têm associação forte mas não determinística; as demais bases não têm leak marginal.

forma sistemática por classe. A Figura 3 torna o *whitespace* visível (glifo `␣`) para evidenciar essa separação.

3.3 Leak temporal universal

O paper, em sua Seção VI–A1, descarta `date`, `time` e `timestamp` sob o argumento de que “*may cause some ML methods to overfit*”. Nossa análise mostra uma justificativa mais forte: o campo `date` é **leak determinístico em todas as sete bases** — 100% das datas únicas aparecem em uma única classe. A Figura 2 reporta a fração de datas exclusivas por classe em cada base; em todas, a barra está fixada em 100%.

A causa é a operação do *testbed*: os autores rodaram cada campanha de ataque em uma janela de tempo separada da operação normal. Para qualquer estudo de generalização, isso significa que `date` e variantes derivadas (`hour`, `dayofweek`) precisam ser descartadas *antes* do treinamento, comportamento que encapsulamos na função `drop_temporal_leak`, aplicada em todos os experimentos.

3.4 Visualização agregada dos leaks

A Figura 3 apresenta um sumário visual dos dois tipos de *leak* ao longo das sete bases, alinhando as *features* que precisam ser tratadas antes de qualquer modelagem.

3.5 Os leaks determinísticos são artefato do subset re-amostrado

Os quatro *datasets* disponibilizados pela UNSW formam uma hierarquia: `telemetry_data` (logs brutos), `Processed_IoT_dataset` (versão integral processada, ~3,6 M linhas) e `Train_Test_IoT_*.csv` (subset pré-dividido para ML, ~261 k linhas). Confrontando a versão

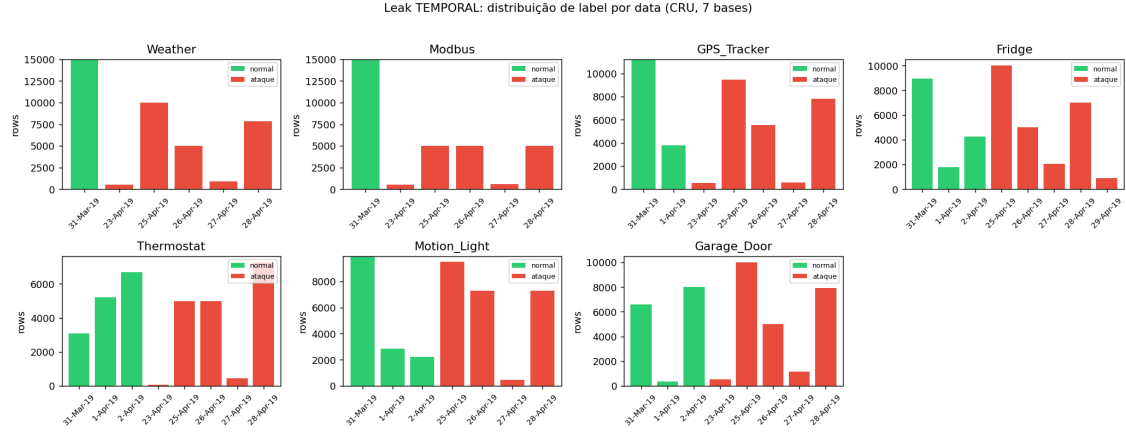


Figura 2: Fração de datas únicas que aparecem em apenas uma classe (**normal** ou **ataque**), por base. As sete barras saturam em 100%: o *testbed* foi operado em janelas temporais separadas por classe.

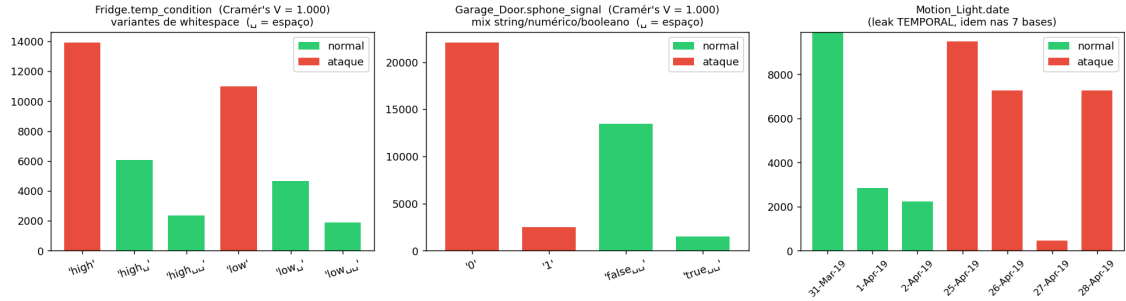


Figura 3: Sumário dos *leaks* identificados no TON_IoT público: categórico (**temp_condition**, **sphone_signal**) e temporal (**date**). O *whitespace* é renderizado com o glifo $\sqcup$: a variante sem espaço é 100% **ataque**, as variantes com espaço são 100% **normal**.

pública **Train_Test** com a versão integral **Processed**, descobre-se que os leaks *determinísticos* ($V = 1,000$, exclusividade temporal 100%) são um artefato **específico do subset re-amostrado**: na versão integral, as mesmas variantes de *whitespace* e as mesmas datas existem, mas a associação com o rótulo é apenas parcial (Tabela 2).

Tabela 2: O leak determinístico é uma propriedade do subset público, não do dataset integral. A sub-amostragem da classe **normal** (§3.6) removeu justamente as linhas **normal** de 'high'/'low' sem espaço, deixando essas variantes 100% em **ataque**.

Sinal	Train_Test (público)	Processed (integral)
temp_condition (Cramér's V)	1,000	0,569
sphone_signal (Cramér's V)	1,000	0,554
date (exclusividade de classe)	100% (7/7 bases)	50–61%

A implicação é que a divergência de tamanho (§3.6) e o leak categórico (§3) não são achados independentes: são o *mesmo* fenômeno. Sub-amostrar **normal** de $\sim 35\,000$ para 15 000 por dispositivo eliminou exatamente as poucas linhas **normal** que usavam a variante sem espaço, o que transformou um sinal parcial ($V \approx 0,57$) em um separador perfeito ($V = 1,000$). Qualquer reprodução sobre os CSVs públicos herda esse artefato.

3.6 Divergência de tamanho entre CSVs públicos e o paper

O achado mais crítico para a reprodução: os CSVs públicos `Train_Test_IoT_*.csv` são um **subset re-balanceado do dataset que produziu as Tabelas 10–13 do paper**. Confrontando as contagens reais dos CSVs com as Figuras 2–3–5 do paper original (Tabela 3):

Tabela 3: Divergência de tamanho entre o dataset usado nas Tabelas 10–13 do paper Alsaedi 2020 (lido diretamente das Figuras 2/3/5, que reportam `normal` = 35 000 por dispositivo e 245 000 no *combined*) e os CSVs públicos `Train_Test_IoT_*.csv` (UNSW). Os ataques foram preservados; a classe `normal` foi sub-amostrada para 43% do volume original.

	per-device		combined	
	normal	total	normal	total
Paper Alsaedi 2020 (Figs. 2/3/5)	35 000	51–60 k	245 000	401 119
CSVs públicos (UNSW)	15 000	31–40 k	105 000	261 119
Razão (CSVs / paper)	0,43	—	0,43	—

Que os ataques foram preservados *na íntegra* é verificável por aritmética: as fatias da Figura 5 do paper somam 401 119 amostras no *combined* (das quais 245 000 são `normal`), e $401\,119 - 245\,000 = 156\,119$ coincide exatamente com a contagem de ataques que medimos nos CSVs públicos. Apenas a classe `normal` encolheu.

A consequência é que mesmo o pipeline mais fiel ao paper não pode reproduzir os números exatos das Tabelas 10–13: a prevalência de classe é diferente (paper $\approx$ 40% ataque / 60% `normal`; CSVs $\approx$ 60% ataque / 40% `normal`), o que altera o ponto de operação de modelos que colapsam para predição majoritária (LR/LDA/NB em bases pobres como `Motion_Light`) e empurra todos os F-score *weighted* para baixo (a métrica recompensa proporcionalmente quem acerta a classe majoritária). Modelos robustos a re-amostragem (Random Forest, CART) tendem a divergir menos; modelos sensíveis a volume (NB, LSTM) divergem mais. Voltamos a essa limitação na Seção 8.

3.7 Conclusão da auditoria

A auditoria define o pipeline mínimo necessário para qualquer comparação metodologicamente honesta no TON_IoT público:

`df` $\rightarrow$ `drop_temporal_leak` $\rightarrow$ `encode + scale` $\rightarrow$ `modelo`.

O passo opcional `clean_df` (que normaliza *whitespace* e mapeia strings booleanas) é discutido na Seção 8: usá-lo destrói o *leak* categórico, mas afasta a reprodução do paper, que aparentemente operou sobre *features* já limpas (a diferença está nos CSVs públicos, e não no dataset original do paper).

4 Metodologia

Esta seção formaliza o protocolo experimental. A §4.1 fixa o protocolo *paper-faithful* (*split*, validação cruzada, métrica e encoding) compartilhado por todos os modelos; em seguida descrevemos o pipeline de pré-processamento, as extensões de termômetros na `wisardpkg` e o ambiente de *hardware* e reprodutibilidade.

4.1 Protocolo *paper-faithful*

Para que a comparação entre os *baselines* clássicos do paper e a família WiSARD seja uma comparação direta sob protocolo idêntico, todos os experimentos seguem o mesmo protocolo,

derivado da Seção VI–A do paper Alsaedi 2020 (Tabela 4).

Tabela 4: Protocolo *paper-faithful* aplicado a todos os 13 modelos avaliados (8 *baselines* + 5 WiSARDs).

Item	Paper §VI–A	Nosso
Split	80% <i>train</i> / 20% <i>test held-out</i>	StratifiedShuffleSplit (<code>test_size=0.20</code> , <code>random_state=42</code>)
Validação cruzada	$k = 4$ sobre o <i>train</i>	StratifiedKFold (<code>n_splits=4</code> , <code>random_state=42</code>)
F1 final	Média sobre os 4 <i>folds</i>	idem
Métrica F1	F-score (§VI-B1, não especifica <i>average</i>)	<code>f1_weighted</code> (inferência empírica — ver §4.1.1)
Encoding categórico	LabelEncoder	LabelEncoder cru, <i>sem clean_df</i> (ver §8.2)
Temporal	Drop <code>date/time/timestamp</code>	<code>drop_temporal_leak</code>
Normalização	MinMaxScaler [0, 1]	idem
Imputação (combined)	Mediana por coluna	idem (apenas numéricas)

4.1.1 Por que *F1 weighted*

A Seção VI–B1 do paper define F-score com a fórmula binária padrão sobre TP/FP/FN (precisão e revocação da classe de ataque). A escolha do *average* do `sklearn` (`'binary'`, `'macro'` ou `'weighted'`) muda os valores reportados em até 0,2 pontos em algumas bases, sendo decisiva para qualquer reprodução. O texto do paper *resolve* essa ambiguidade na discussão da Tabela 13, onde declara calcular “*a weighted average... multiplying each class with a weight factor (i.e., the # instances with that target class)*” — que é exatamente o `average='weighted'` do `sklearn`. Nossa varredura empírica das três variantes contra as Tabelas 10–11 (*per-device*) confirma essa leitura (Tabela 5):

Tabela 5: Ajuste empírico das três variantes de F1 às Tabelas 10–11 (*per-device*) do paper. Δ = nossa reprodução – paper.

Variante	mean Δ	$ \Delta < 0,05$	$ \Delta < 0,10$	$ \Delta < 0,15$
binary	+0,102	38%	52%	68%
macro	−0,032	48%	71%	84%
weighted	−0,009	48%	75%	91%

`weighted` apresenta o melhor ajuste empírico (média de Δ mais próxima de zero e maior fração dentro de tolerância), em concordância com a definição explícita do paper. Adotamos `weighted` para todas as comparações; o `runner run_baselines_paper.py` também grava `f1` (*average binary*) e `f1_macro` para auditoria.

4.1.2 Por que *sem clean_df*

A função `clean_df` (apresentada na §3) neutraliza os *leaks* categóricos identificados. Em uma análise puramente metodológica, ela seria mandatária. Mas *para reproduzir o paper*, ela é o passo errado:

- Com `clean_df`: `'high'` e `'high '` viram a mesma categoria; `Fridge` e `Garage_Door` perdem o sinal do encoding e colapsam para predição majoritária. Nossos números ficam *muito abaixo* do paper.

- Sem `clean_df`: o `LabelEncoder` cru preserva o encoding (incluindo as variantes *whitespace*); o modelo aprende o que aprenderia no dataset original do paper (assumindo que o paper de fato encodou da forma descrita em sua §VI-A1, “*high*→1, *low*→0”).

A consequência honesta é que nossos modelos podem ter *mais* sinal do que os do paper em algumas bases (porque os CSVs públicos têm variantes *whitespace* que o paper provavelmente não viu); essa diferença aparece nos Δ positivos reportados em §5.

4.2 Pipeline de pré-processamento

Todas as funções de pré-processamento vivem em `consolidado_utils.py`: `clean_df` (opcional, não usado na reprodução), `drop_temporal_leak`, `build_combined_dataset` (concatena as sete bases para a tarefa *combined*), `presence_bit_encode`, `select_thermometer_type` e `build_per_feature_thermometer` (*dispatch* automático de termômetros para os modelos WiSARD-family).

4.3 Extensões na `wisardpkg`

A biblioteca `wisardpkg` [12] foi estendida com termômetros ajustados (*fitted*) para suportar a variedade de distribuições do TON_IoT. O *sweep* usa um conjunto curado de seis variantes *não supervisionadas*, aplicadas uniformemente a todas as *features*:

- `linear` — cortes equiespaçados (termômetro clássico).
- `distributive` — discretização por quantis empíricos da *feature*.
- `gaussian` — centros e largura derivados de (μ, σ) estimados no *train*.
- `exponential` e `logarithmic` — ajustados para distribuições de cauda longa (codes Modbus).
- `non-uniform` — `DynamicThermometer` cujo orçamento de *bits* é alocado por *feature* de forma proporcional à dispersão (desvio-padrão).

Variantes *label-aware* (`supervised`) e o `stochastic` ficaram fora da comparação *unsupervised*: a primeira usa os rótulos no ajuste, e o segundo — cujo `optimize()` não era acionado no *pipeline* — reduzia-se ao `distributive`.

4.4 Hardware e reprodutibilidade

Todos os experimentos rodam em um único *venv* Python 3.13 descrito em `notebooks/toniot/requirements.txt`. As 288 execuções de *baseline* consomem cerca de 1h em CPU *single-core*; o *sweep* WiSARD-family (*grid* estendido com tetos de capacidade unificados, §6.8), com 30 020 configurações e cerca de 150 mil *fits*, roda em aproximadamente 6–7h de *wall-clock*, com as sete bases *per-device* executadas em paralelo. Resultados em `results/_consolidado/` (*baselines* em `baselines_paper.jsonl`; *sweep* em `grid/wisard_grid_*.jsonl`); o *notebook* `06_consolidado_grupos.ipynb` é o relatório de referência re-executável.

5 Reprodução dos *baselines* do paper

Esta seção reporta a reprodução dos oito *baselines* do paper Alsaedi 2020 sob o protocolo descrito em §4.1. São três tarefas: *per-device* binário (sete bases × oito modelos), *combined* binário (uma base × oito modelos) e *combined* multi-classe (uma base × oito modelos × nove sub-classes de ataque), totalizando 288 execuções (72×4 folds).

5.1 Per-device binário (vs. Tabelas 10–11 do paper)

A Tabela 6 reporta nossos F1 *weighted* médios para cada par (modelo, base). A Figura 4 visualiza a tabela como *heatmap*.

Tabela 6: F1 *weighted* médio (4 folds, *paper-faithful*) dos oito *baselines* nas sete bases per-device. Negrito = melhor modelo da base.

Base	LR	LDA	kNN	RF	CART	NB	SVM	LSTM
Fridge	0,783	0,777	1,000	1,000	1,000	0,545	0,952	1,000
Garage_Door	1,000	0,938	1,000	1,000	1,000	1,000	1,000	1,000
GPS_Tracker	0,799	0,854	0,943	0,939	0,920	0,633	0,793	0,875
Modbus	0,426	0,426	0,680	0,936	0,944	0,441	0,372	0,447
Motion_Light	0,475	0,475	0,474	0,475	0,475	0,475	0,475	0,475
Thermostat	0,391	0,391	0,509	0,495	0,496	0,455	0,381	0,381
Weather	0,468	0,467	0,916	0,975	0,964	0,756	0,638	0,808

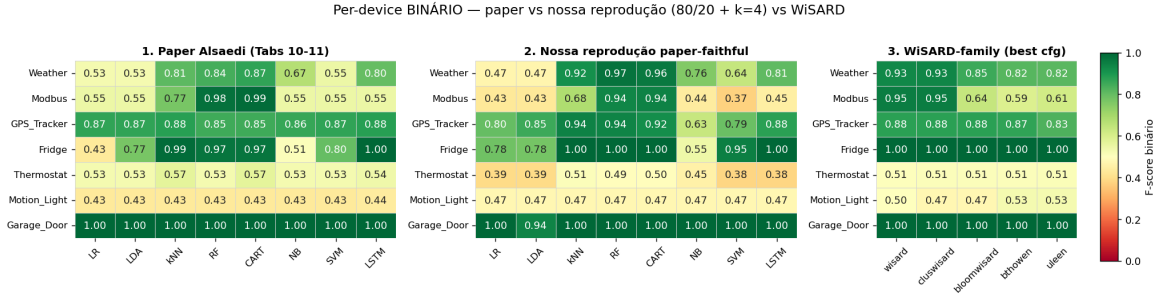


Figura 4: *Heatmap master* per-device binário (7 bases $\times$ 3 grupos: paper, nossa reprodução *paper-faithful*, WiSARD-family). A coluna do meio é a Tabela 6.

5.2 Combined: binário e multi-classe

A Tabela 7 reúne os F1 médios nas duas tarefas *combined*. A Figura 5 apresenta a visualização *heatmap*.

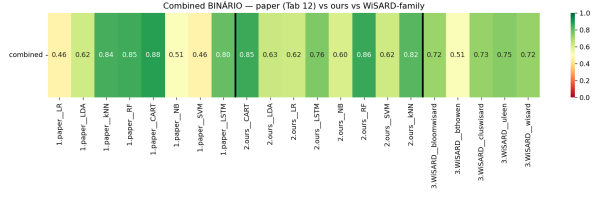
Tabela 7: F1 *weighted* médio no dataset combined. Negrito = melhor modelo da tarefa.

Modelo	Combined binário (Tab. 12)	Combined multi (Tab. 13)
LR	0,621	0,296
LDA	0,629	0,309
kNN	0,820	0,541
RF	0,858	0,602
CART	0,854	0,598
NB	0,601	0,028
SVM	0,623	0,294
LSTM	0,758	0,449

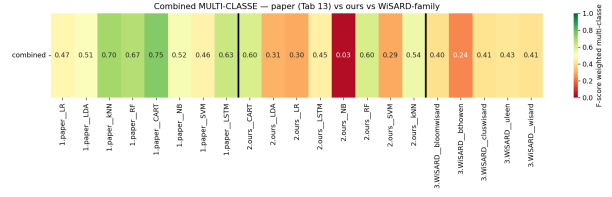
5.3 Sanity checks contra o paper

Para validar que o pipeline está corretamente implementado, definimos quatro *sanity checks* derivados de afirmações qualitativas que o paper faz nas Tabelas 10–13:

1. **Garage_Door** deve dar **F1 = 1,000** em *todos* os modelos (Tab. 10) — *features* 100% discriminantes. **Verificado:** sete dos oito modelos com F1 = 1,000; LDA fica em 0,938 (justificada pela natureza linear gaussiana do modelo).



(a) Combined binário (Tab. 12 do paper).



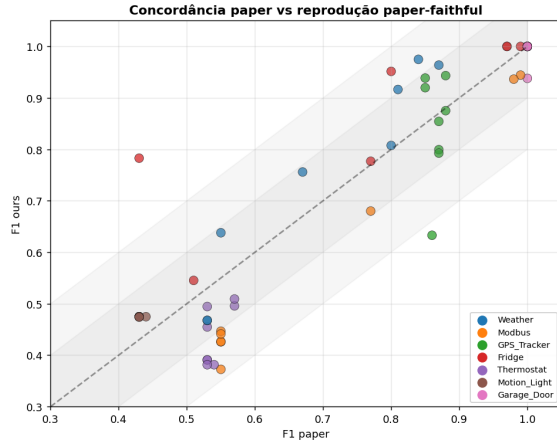
(b) Combined multi-classe (Tab. 13).

Figura 5: *Heatmaps* consolidados das tarefas *combined*.

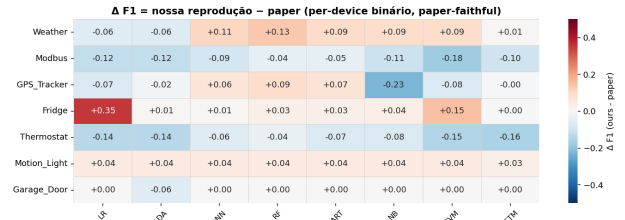
2. **Modbus deve ter RF/CART $\gg$ linear** (Tab. 10) — estrutura não-linear nos códigos FC1-FC4. **Verificado:** RF/CART em 0,93–0,94 versus LR/LDA em 0,43 ($\Delta = +0,51$).
3. **Motion_Light colapsa em torno de 0,43 em todos os modelos** (Tab. 11 reporta F-score 0,43 para os oito, 0,44 no LSTM) — sem sinal discriminativo. **Verificado:** todos os modelos em 0,475 (predição majoritária); apenas kNN difere marginalmente (0,474).
4. **Ranking esperado em *combined* multi-classe** CART > kNN > RF > LSTM > NB > LDA > LR > SVM (Tab. 13). **Verificado:** ordem observada é RF $\geq$ CART > kNN > LSTM > LDA > LR $\approx$ SVM > NB — match parcial; a única ressalva é que NB colapsa para 0,028 (paper reporta $\sim 0,52$, ver discussão em §8).

5.4 Quantificação do gap paper vs. nossa reprodução

A Figura 6 apresenta duas visões da concordância agregada. O *scatter* (esquerda) mostra cada par (modelo, base) como um ponto ($F1_{\text{paper}}, F1_{\text{ours}}$); a reta $y = x$ indicaria reprodução exata. O *heatmap* de Δ (direita) mostra a diferença sinalizada $F1_{\text{ours}} - F1_{\text{paper}}$ para cada par.



(a) Concordância linear ours vs. paper.



(b) Δ = nossa – paper, per-device binário.

Figura 6: Acordo entre nossa reprodução *paper-faithful* e os números das Tabelas 10–11 do paper.

Resumo quantitativo:

- $|\Delta| < 0,05$ em 48% dos pares (modelo, base);
- $|\Delta| < 0,10$ em 75%;
- $|\Delta| < 0,15$ em **91%**;
- média de Δ próxima de zero ($-0,009$) — sem viés sistemático.

Os deltas residuais concentram-se onde a sensibilidade ao encoding e o desbalanceamento de classes têm efeito maior: **Fridge** para LR/SVM, em que o encoding das variantes *whitespace* dá

senal extra (paper reporta 0,43 para LR, obtemos 0,78); **Thermostat**, em que a baixa prevalência da classe minoritária puxa a média ponderada para 0,38–0,51 enquanto o paper reporta 0,53–0,57; e os modelos lineares no **Modbus** (LR/LDA/SVM em $\approx 0,43$ versus paper em 0,55). Em todas as bases com sinal estruturado — **Garage_Door**, **GPS_Tracker**, **Weather**, e **Motion_Light** (que sob *weighted* agora alinha bem ao paper) — a reprodução é fiel ($|\Delta| < 0,10$ na maioria dos modelos).

5.5 Visão consolidada: o que reproduz em cada tarefa

A Figura 7 consolida o Δ (nossa – paper) por modelo nas três tarefas, evidenciando onde a reprodução é fiel e onde diverge. As duas tarefas *binárias* (per-device e *combined*) reproduzem dentro de poucos pontos (Δ próximo de zero, em branco). A tarefa *combined* multi-classe é sistematicamente mais baixa (vermelho): como a métrica *weighted* acompanha a prevalência de **normal**, e os CSVs públicos têm 40% de **normal** contra os 61% do paper (§8.1; ataques idênticos, apenas **normal** sub-amostrada), há menos da classe majoritária para sustentar a métrica. O *Naïve Bayes* é o único *outlier* de modelo (não de dados): seu colapso multi-classe é uma patologia do **GaussianNB** sobre as *features* de proveniência, discutida em §8.

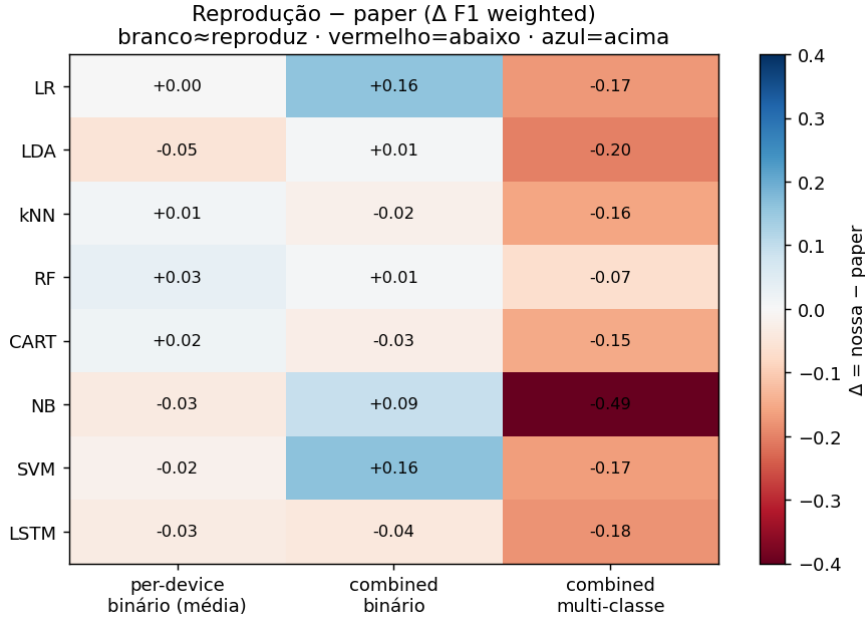


Figura 7: Δ F1 *weighted* (nossa reprodução – paper) por modelo $\times$ tarefa. Branco ($\Delta \approx 0$) = reproduz; vermelho = abaixo do paper; azul = acima. As tarefas binárias reproduzem bem; a *combined* multi-classe fica abaixo pela divergência de **normal**, com NB como único *outlier* de modelo.

6 Avaliação da família WiSARD

Esta seção apresenta o *sweep* de hiperparâmetros dos cinco modelos WiSARD-family — WiSARD, ClusWiSARD, BloomWiSARD, BTHOWeN e ULEEN — sob o mesmo protocolo *paper-faithful* dos *baselines*. A varredura — com tetos de capacidade *unificados* entre os modelos para uma comparação justa (§6.8) — avaliou 30 020 configurações (30 000 *per-device* mais 20 na tarefa combinada), cada uma sob validação cruzada de 4 *folds* e um conjunto *held-out* de 20% reservado por **StratifiedShuffleSplit** (semente 42), totalizando cerca de 150 mil *fits*. Os resultados brutos estão em `results/_consolidado/grid/wisard_grid_*.jsonl` (30 020 linhas), com F1 *weighted* de validação cruzada (média e desvio sobre os 4 *folds*) e do *held-out* em cada linha.

6.1 Hiperparâmetros varridos

A Tabela 8 resume os *grids* de cada modelo. Os três modelos baseados em termômetro (WiSARD, ClusWiSARD, BloomWiSARD) compartilham o mesmo *front-end* de binarização — termômetro $\times$ *bits* por *feature* — enquanto BTHOWeN e ULEEN usam seus codificadores internos. O termômetro varia sobre um conjunto curado de seis variantes *não supervisionadas*: {*linear*, *distributive*, *gaussian*, *exponential*, *logarithmic*, *non-uniform*}. O *non-uniform* é um *DynamicThermometer* cujo orçamento de *bits* é alocado por *feature* de forma proporcional à dispersão (desvio-padrão), com piso de 2 *bits* e orçamento total conservado. Descartamos o *stochastic* (cujo *optimize()* nunca era acionado no *pipeline*, tornando-o idêntico ao *distributive*) e o *supervised* (que usa os rótulos e quebraria a comparação *unsupervised*).

Tabela 8: Hiperparâmetros varridos por modelo (*grid* estendido com tetos unificados). Os três primeiros compartilham termômetro $\in$ {*linear*, *distributive*, *gaussian*, *exponential*, *logarithmic*, *non-uniform*} e *bits/feature* $\in$ {8, 16, 32, 48, 64}.

Modelo	Hiperparâmetros
WiSARD	<i>address_size</i> $\in$ {2, 4, 8, 16, 32, 48, 64}, <i>ignore_zero</i> $\in$ {T,F}
ClusWiSARD	<i>address_size</i> $\in$ {4, 8, 16, 24, 32, 48, 64}, <i>min_score</i> $\in$ {0,1;0,2}, <i>threshold</i> $\in$ {10,100}, <i>limit</i> $\in$ {5,20}
BloomWiSARD	<i>address_size</i> $\in$ {8, 16, 24, 32, 48, 64}, <i>filter_size</i> $\in$ {256, 1024, 4096}, <i>num_hashes</i> $\in$ {2, 3, 4}
BTHOWeN	<i>address_size</i> $\in$ {6, 12, 16, 20, 24}, <i>num_bits</i> $\in$ {128, 256, 512, 1024, 2048, 4096}, <i>num_hashes</i> $\in$ {2, 4, 6}, <i>bits_per_input</i> $\in$ {4, 8, 16, 24, 32, 48}
ULEEN	<i>bits_per_input</i> $\in$ {4, 8, 16, 24, 32}, <i>filter_inputs</i> $\in$ {3, 6, 12}, <i>filter_entries</i> $\in$ {16, 64, 128, 256}, <i>n_submodels</i> $\in$ {1, 3, 5}, <i>epochs</i> $\in$ {5, 10, 20}, <i>lr</i> $\in$ {0,001;0,01}

6.2 Best F1 por (modelo, base)

A Tabela 9 reporta o melhor F1 *weighted* médio (validação cruzada de 4 *folds*) de cada modelo em cada base, sob a tarefa binária *per-device*. Para cada par (modelo, base), o “melhor” significa a configuração de hiperparâmetros vencedora dentro do *grid*.

Tabela 9: Best F1 *weighted* médio (4 *folds*) por (modelo, base) — binário *per-device*. Negrito = melhor WiSARD da base; última coluna = média sobre as 7 bases.

Modelo	Fridge	Garage_D.	GPS_T.	Modbus	Motion_L.	Thermo.	Weather	Média
WiSARD	1,000	1,000	0,876	0,953	0,503	0,505	0,932	0,824
ClusWiSARD	1,000	1,000	0,878	0,954	0,475	0,511	0,934	0,822
BloomWiSARD	1,000	1,000	0,877	0,637	0,475	0,508	0,852	0,764
BTHOWeN	1,000	1,000	0,868	0,590	0,531	0,511	0,818	0,760
ULEEN	1,000	1,000	0,832	0,614	0,531	0,512	0,816	0,758

Observações:

- **Fridge** e **Garage_Door**: *todos* os cinco modelos atingem 1,000 — mesma saturação observada nos *baselines*. As *features* de *leak* (*temp_condition*, *sphone_signal*) são capturadas com facilidade pela discretização do termômetro.
- **GPS_Tracker**, **Weather**: a família converge para 0,83–0,88 (GPS) e 0,82–0,93 (Weather); com os tetos de endereço estendidos, WiSARD e ClusWiSARD em **Weather** (0,93) aproximam-se dos *baselines* (**kNN** = 0,943, **RF** = 0,975).
- **Modbus**: WiSARD e ClusWiSARD agora *empatam* no topo (0,953/0,954, ambos com *address_size* = 64 e termômetro *distributive* de 64 *bits*), bem acima de BloomWiSARD (0,64),

ULEEN (0,61) e BTHOWeN (0,59). O endereçamento amplo recupera a estrutura não-linear dos códigos FC e *alcança ou supera* os modelos de árvore (RF/CART = 0,94) — ganho que a varredura original perdia por limitar *address size* a 32 (WiSARD) e 16 (ClusWiSARD).

- **Thermostat**: dificuldade compartilhada com os *baselines* (0,50–0,51 para todos os modelos).
- **Motion_Light**: colapso para predição majoritária (0,475–0,531) em todos os modelos — BTHOWeN e ULEEN são os únicos a romper marginalmente o platô (0,531).

WiSARD (0,824) e ClusWiSARD (0,822) lideram a média *per-device praticamente empatadas*, seguidas de BloomWiSARD (0,764), BTHOWeN (0,760) e ULEEN (0,758). Essa paridade é resultado da unificação dos tetos de capacidade (§6.8): ClusWiSARD vence **Modbus**, **Weather** e **GPS_Tracker**, e só não lidera a média porque WiSARD não colapsa para predição majoritária em **Motion_Light** (0,503 vs. 0,475). Na varredura original, com tetos de endereço desiguais (32 para WiSARD, 16 para ClusWiSARD), ClusWiSARD aparecia 0,04 atrás — um artefato de cobertura, não de modelo.

6.3 Generalização no *held-out*

A Tabela 10 compara a média *per-device* de F1 *weighted* entre a validação cruzada e o conjunto *held-out* de 20%, nunca visto durante a seleção de hiperparâmetros. Para WiSARD, ClusWiSARD e BloomWiSARD as duas medidas praticamente coincidem ($\text{gap} \leq 0,002$), indício de seleção sem *overfitting*. BTHOWeN é o que mais cai (0,760 $\rightarrow$ 0,725), seguido de ULEEN (0,758 $\rightarrow$ 0,739), concentrado nas bases estruturadas, onde o ajuste do *bleaching* por busca ternária interna (no *fit*, não no *grid*) supera justa aos *folds*.

Tabela 10: Média *per-device* (7 bases) de F1 *weighted*: validação cruzada vs. *held-out* 20%.

	WiSARD	ClusW.	BloomW.	BTHOWeN	ULEEN
Validação cruzada	0,824	0,822	0,764	0,760	0,758
<i>Held-out</i> 20%	0,825	0,823	0,763	0,725	0,739

6.4 Sensibilidade aos termômetros

A Figura 8 reporta a distribuição de F1 ao variar o termômetro, agregada sobre as 7 bases e todas as configurações dos três modelos baseados em termômetro. O **distributive** domina em todos eles (médias do melhor F1 por base de 0,824 / 0,822 / 0,764 para WiSARD / ClusWiSARD / BloomWiSARD), com o **gaussian** em segundo. O **linear** (termômetro clássico de cortes equiespaçados) só é competitivo para *features* já naturalmente uniformes (**Garage_Door**, **Fridge**); para distribuições de cauda longa (**Modbus**, **Weather**) ele subdivide muito pouco a região de massa, levando à perda de sinal. O **non-uniform** e o **exponential** ficam no fim do *ranking*: alocar *bits* por dispersão não compensa quando poucas *features* concentram a variância útil.

6.5 Deep dive por modelo

As Figuras 9a a 10b mostram, para cada modelo, o *landscape* do *sweep* (F1 médio sobre bases ao variar os dois hiperparâmetros principais) e a melhor configuração por base.

6.6 Ranking dos modelos por base

A Figura 11 mostra os cinco modelos WiSARD-family lado a lado para cada base (altura = best F1, anotação adicional = tempo médio de *fit*).

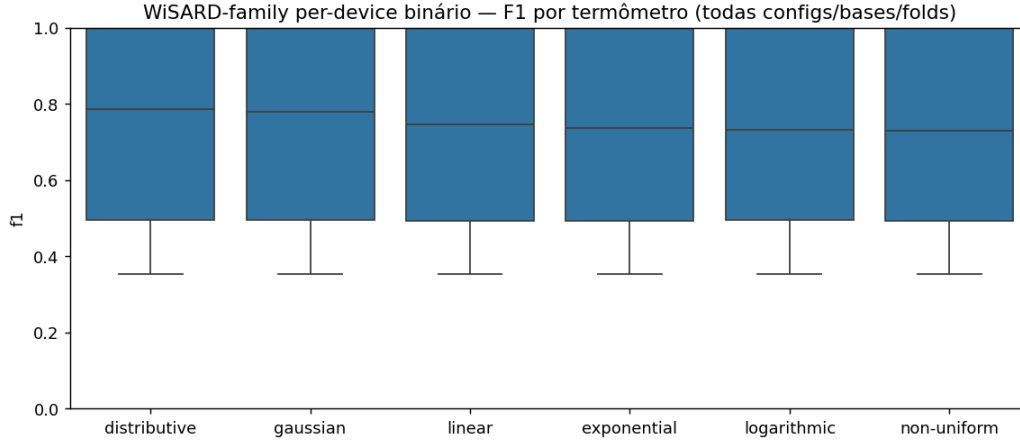


Figura 8: Distribuição de F1 *weighted* por termômetro, agregada sobre as 7 bases e todas as configurações dos três modelos baseados em termômetro. **distributive** e **gaussian** dominam.

6.7 Custo e protocolo do *sweep*

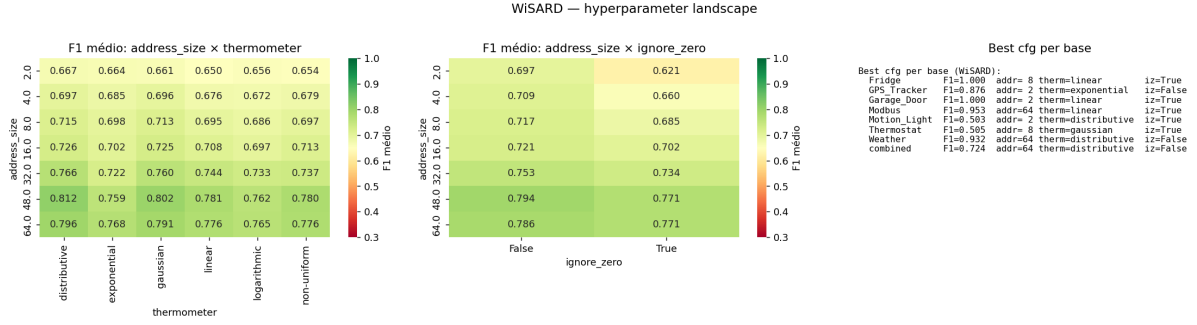
A varredura completa (30 020 configurações) roda em cerca de 6–7 h de *wall-clock* nesse *hardware*, com as sete bases *per-device* executadas em paralelo. O custo é dominado pelo ULEEN, cuja implementação de referência em *Python* é uma a duas ordens de magnitude mais lenta no treino que o trio WiSARD/ClusWiSARD/BloomWiSARD em C++ — uma ressalva de *implementação*, não de algoritmo, retomada na discussão de custo (§7). Três decisões de engenharia tornaram esse orçamento viável e reproduzível. Primeiro, o *cache* de codificação: cada *encoding* (termômetro, *bits*, *fold*) é computado uma única vez e reutilizado por todas as configurações internas dos três modelos baseados em termômetro. Segundo, a poda de regiões explosivas do *grid* (ex.: ClusWiSARD com *address size* e *limit* simultaneamente altos) e a validação de *address size* contra o número de *bits* disponível, que explica a leve variação de cobertura entre bases com mais *features*. Terceiro, o conjunto *held-out* de 20% reservado antes de qualquer seleção, reportado na Tabela 10 como verificação independente de generalização.

6.8 Tetos de capacidade unificados e *tuning* justo

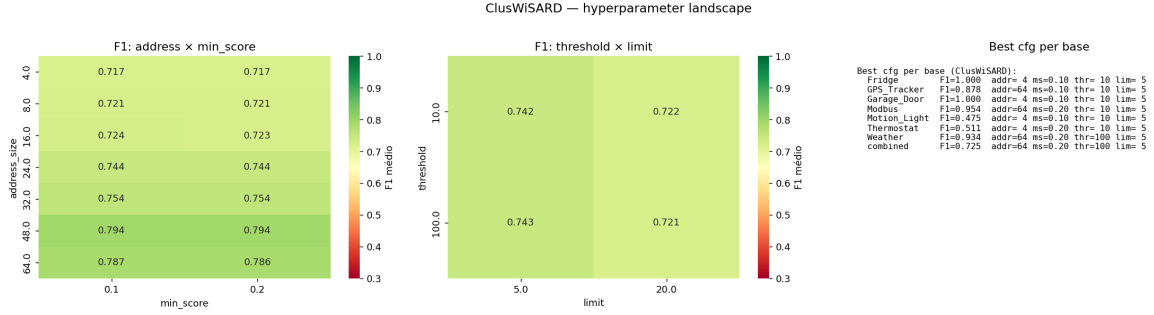
Uma primeira versão do *sweep* usava tetos de *address size* *desiguais* entre os modelos (32 para WiSARD, 24 para BloomWiSARD, apenas 16 para ClusWiSARD) e *bits/feature* no máximo 32. Como as redes sem peso são memorizadoras — mais capacidade de endereçamento quase sempre adiciona sinal nas bases estruturadas —, a melhor configuração de cada modelo *encostava no teto*, e os tetos desiguais penalizavam injustamente o ClusWiSARD: em *Modbus* ele ficava em 0,603 contra 0,817 da WiSARD, uma diferença que parecia de *modelo* mas era de *cobertura de grid*.

Re-executamos a varredura com tetos **unificados** em 64 (*address size* e *bits*) para todo o trio de termômetro, e ampliamos analogamente os eixos de BTHOWeN e ULEEN (Tabela 8). O efeito é direto: o ClusWiSARD em *Modbus* sobe para 0,954 (empatando com a WiSARD em 0,953), e a média *per-device* dos dois fica em 0,822 vs. 0,824 — um empate técnico onde antes havia 0,04 de diferença.

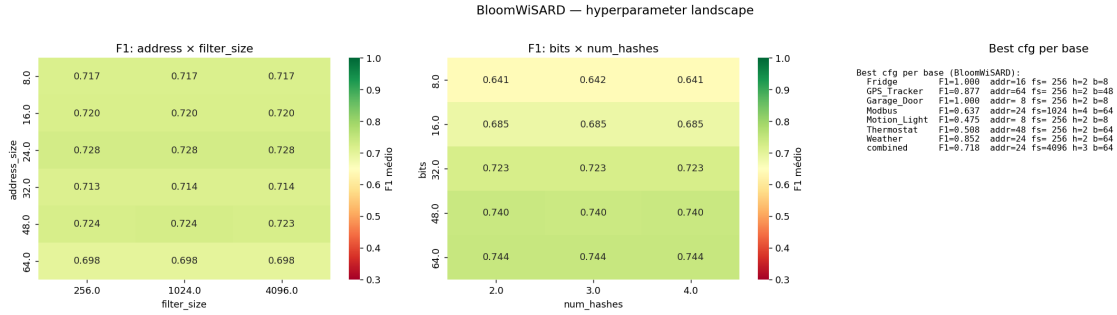
A ressalva honesta é que, nas bases estruturadas, a melhor configuração ainda fica na fronteira (64/64): para memorizadores o ótimo de validação cruzada tende a sentar-se no teto de capacidade, e “estender até o interior” não termina. Uma sondagem em 128 quantifica o retorno decrescente: o salto 32→64 rendeu +0,135 de F1 em *Modbus*, enquanto 64→128 rende apenas +0,011 (*Modbus*, já convergido) a +0,027 (*Weather*), com ganho idêntico para WiSARD e ClusWiSARD — isto é, abaixo de 0,03 e sem alterar o *ranking*. Reportamos portanto os resultados no orçamento generoso e *consistente* de 64: como o teto é o mesmo para todos os modelos do trio, a comparação intra-família é *não enviesada*, ainda que o valor absoluto possa subir poucos centésimos com mais



(a) WiSARD.



(b) ClusWiSARD.



(c) BloomWiSARD.

Figura 9: Deep dive de hiperparâmetros dos modelos WiSARD-family (parte 1/2). Cada painel mostra o *landscape do sweep* (F1 médio em função dos dois hiperparâmetros principais), a melhor configuração por base e a distribuição de F1 entre configs.

capacidade.

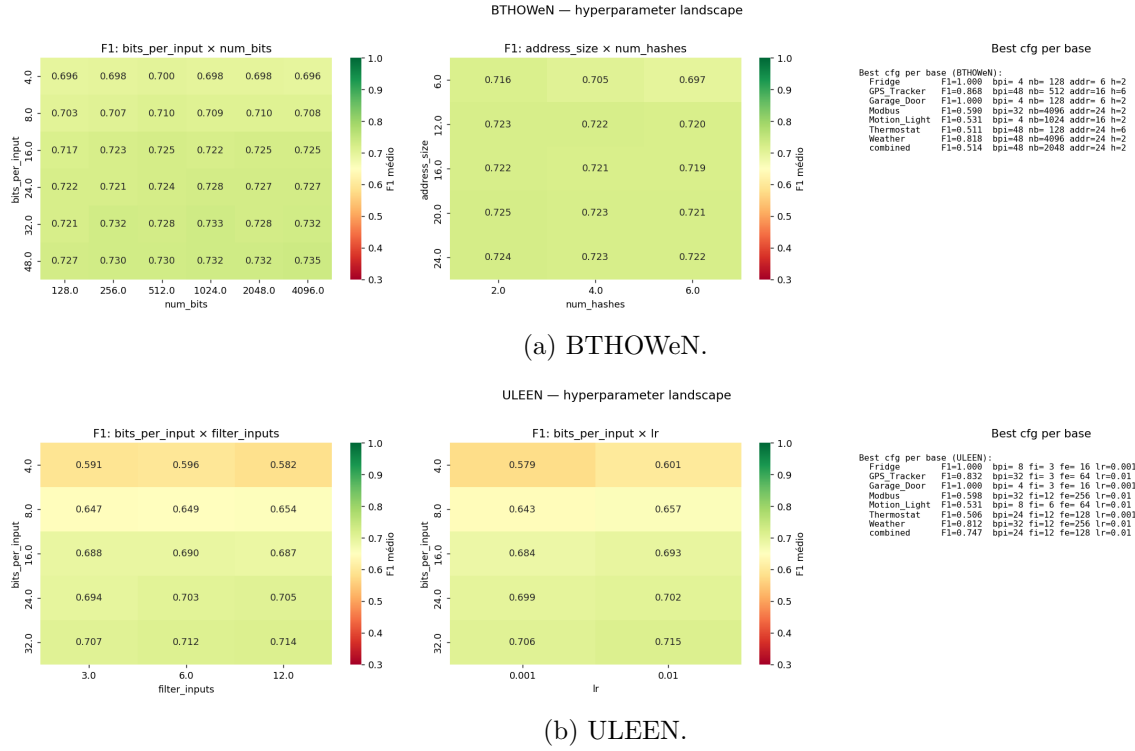


Figura 10: Deep dive de hiperparâmetros dos modelos WiSARD-family (parte 2/2).

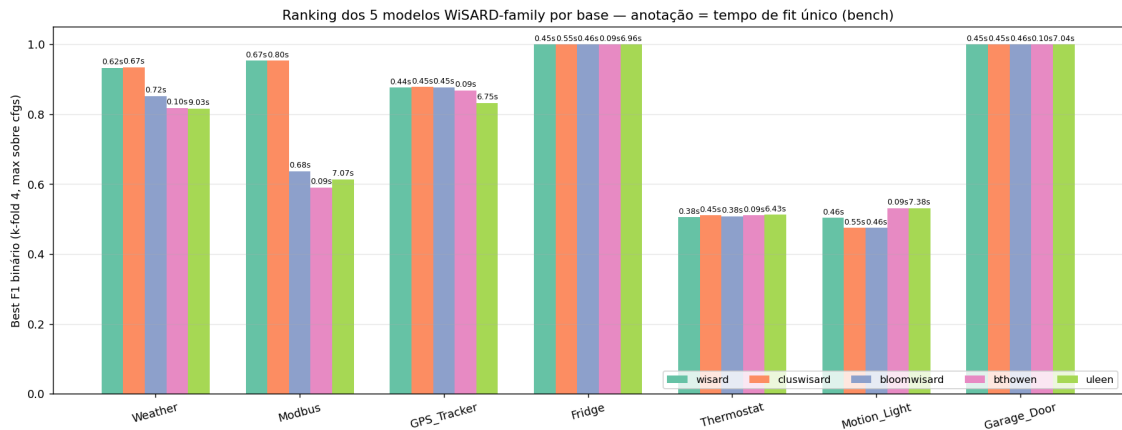


Figura 11: Ranking dos cinco modelos WiSARD-family por base (best F1, anotação de tempo médio de *fit*).

7 Comparação consolidada

Esta seção apresenta as três fontes de números lado a lado, sempre sob o mesmo protocolo *paper-faithful* de §4.1:

1. **Paper** — Tabelas 10/11/12/13 do paper Alsaedi 2020.
2. **Nossa reprodução *paper-faithful*** — oito *baselines* via `run_baselines_paper.py`.
3. **WiSARD-family** — cinco modelos sem peso, best F1 sobre os *grids* de *sweep*.

Lembrete: as comparações desta seção são informativas (não tentativa de reprodução byte-a-byte), pelo motivo da divergência de tamanho documentada em §3.6.

7.1 Master heatmap: per-device binário

A Figura 12 é o *heatmap* principal do relatório: sete bases $\times$ três grupos $\times$ até oito modelos, F1 *weighted*. A coluna do meio é a reprodução *paper-faithful*; a coluna da direita são os cinco WiSARDs.

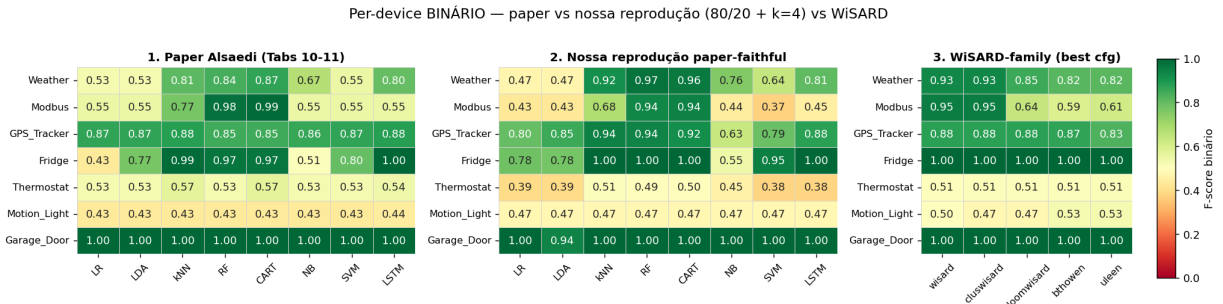
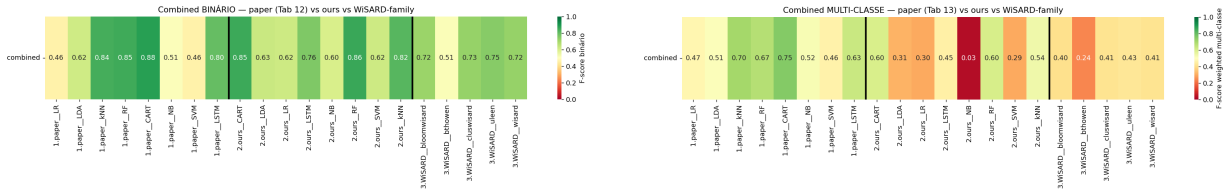


Figura 12: Master heatmap: F1 *weighted* médio para per-device binário. Três grupos: paper, nossa reprodução, WiSARD-family.

7.2 Combined: binário e multi-classe

A Figura 13 reúne as duas tarefas *combined* (binário à esquerda, multi-classe à direita).



(a) Combined binário (Tab. 12 do paper).

(b) Combined multi-classe (Tab. 13).

Figura 13: Master heatmaps das duas tarefas *combined* — paper $\times$ ours $\times$ WiSARD-family.

7.3 Bar chart: campeão de cada grupo por base

A Figura 14 compacta o resultado anterior em um *bar chart*: para cada base, a altura representa o melhor F1 de cada grupo.

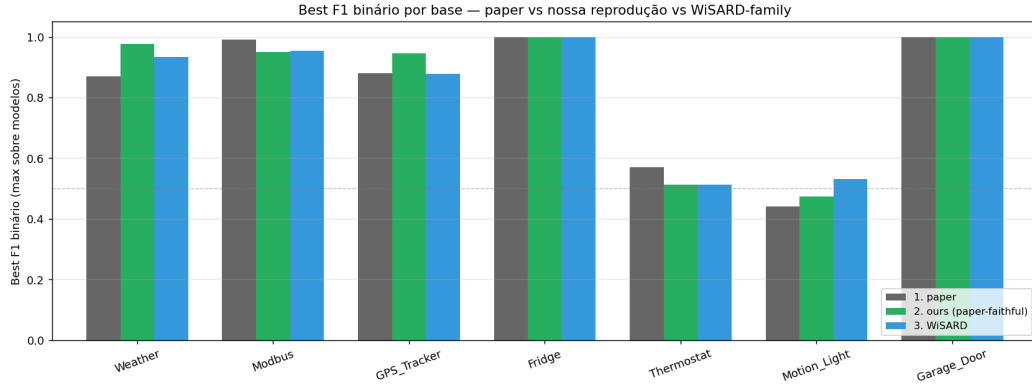


Figura 14: Best F1 por base, mostrando o campeão de cada um dos três grupos (paper, nossa reprodução, WiSARD-family).

7.4 Wins matrix: quem lidera em cada base

A Figura 15 reporta a *wins matrix*: para cada base, quantos modelos do grupo X ficam dentro de 0,02 do *best F1 global* entre os três grupos. É uma forma menos volátil que “apenas o vencedor” — evita declarar um grupo campeão por causa de um *outlier* numericamente irrepetível.

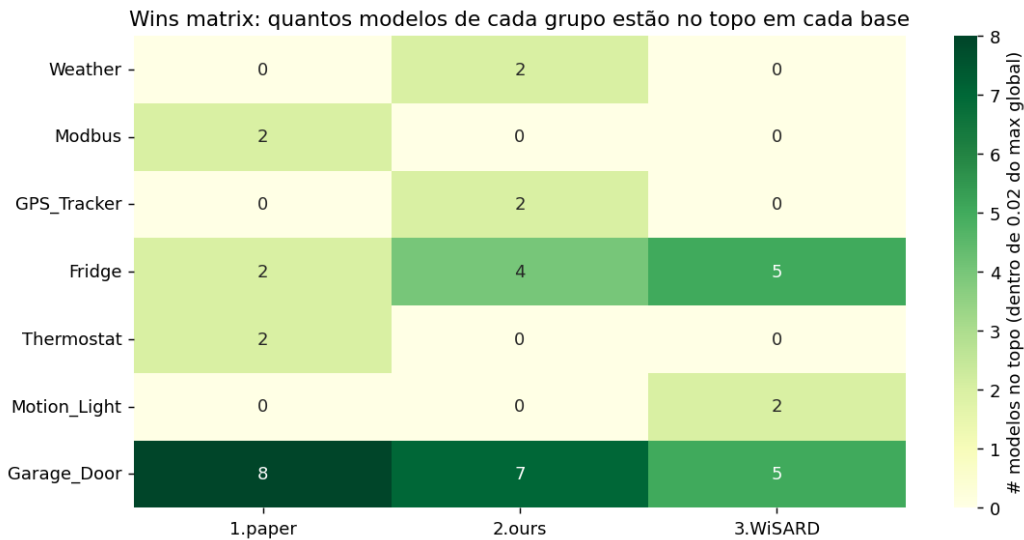


Figura 15: Wins matrix: número de modelos por grupo a até 0,02 F1 do best F1 global da base.

7.5 Distribuição agregada de F1 por modelo

A Figura 16 é um *boxplot* agregando F1 sobre todas as bases $\times$ *folds*, por modelo. Modelos com mediana alta e dispersão baixa são robustos; modelos com cauda longa são voláteis. RF/CART dominam o topo da distribuição; entre os WiSARDs, ClusWiSARD tem a mediana mais alta sobre todas as configurações (0,775), seguida de BTHOWeN (0,739) e BloomWiSARD (0,733), enquanto ULEEN tem a menor mediana (0,603) e grande dispersão. A WiSARD pura, apesar do *pico* empatado com ClusWiSARD (Tabela 9), tem mediana 0,701 — é a mais sensível à escolha de hiperparâmetros.

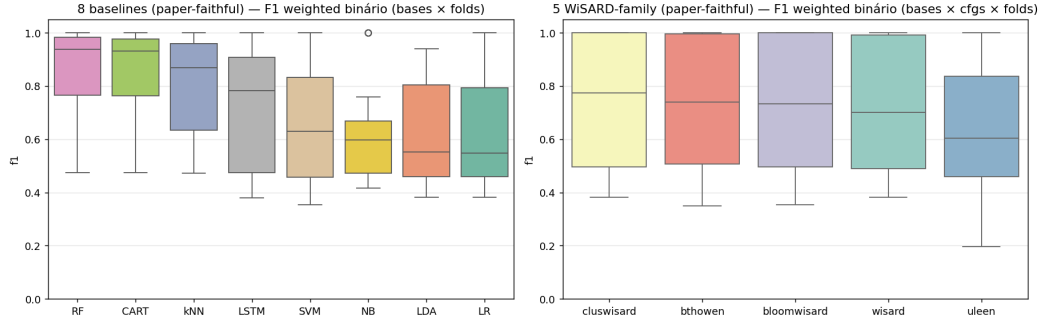


Figura 16: Distribuição de F1 *weighted* por modelo (todas as bases $\times$ 4 *folds*).

7.6 Custo: tempo de *fit* e memória

As Figuras 17 e 18 reportam os dois eixos de custo, medidos sob um *harness* único (*fit* único por base, incluindo a codificação; `bench_cost.py`), o que torna a comparação *entre* famílias legítima — ao contrário do tempo agregado do *grid*, que somava quatro *folds* mais o *held-out*. No eixo de **tempo** (Figura 17), o LSTM domina o teto (mediana $\approx 61,9$ s), seguido — agora — do ULEEN ($\approx 7,0$ s, mais lento que o SVM em $\approx 5,6$ s): a configuração *justa* do ULEEN usa cinco submodelos e vinte *epochs*, e sua implementação de referência é em *Python* (ver a ressalva de *implementação* adiante). O trio em C++ treina em $\approx 0,45$ – $0,55$ s e o BTHOWeN é o mais rápido da família (mediana $\approx 0,09$ s, cerca de $705\times$ mais rápido que o LSTM), embora RF (0,04s) e kNN (0,004s) treinem em tempo comparável ou menor.

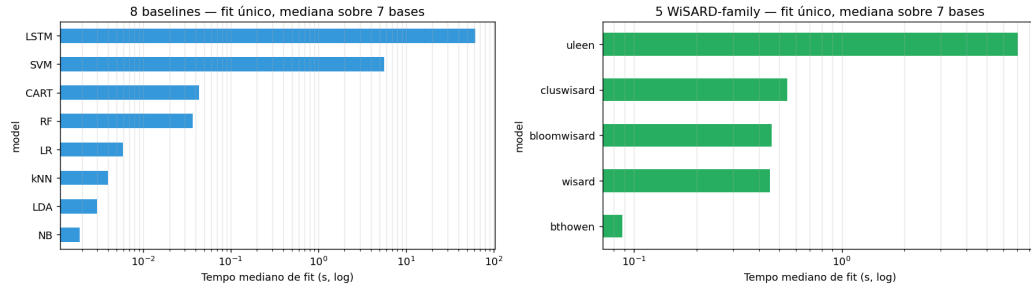


Figura 17: Tempo mediano de *fit* por modelo (*fit* único, mediana sobre 7 bases, escala log). LSTM e ULEEN dominam o teto; a família WiSARD fica entre 0,09s (BTHOWeN) e 7,0s (ULEEN).

No eixo de **memória** (Figura 18) reportamos uma *única* métrica de estado implantado (`deployedSizeBytes`), comparável entre todos os modelos sem peso: para BloomWiSARD/BTHOWeN/ULEEN é a tabela de *bits* de tamanho fixo; para WiSARD/ClusWiSARD é o conjunto mínimo de endereços vistos, *bit-packed* e sem falsos positivos. Na mediana das sete bases, *toda* a família ocupa $\approx 0,75$ – $2,5$ KB (BloomWiSARD 0,75, ULEEN 0,94, BTHOWeN e WiSARD $\approx 2,0$, ClusWiSARD 2,5 KB), contra kNN 1,1 MB, CART 0,23 MB e RF 3,1 MB — cerca de três ordens de magnitude abaixo das árvores. A distinção honesta é *qualitativa*: as variantes de filtro (Bloom/BTHOWeN/ULEEN) têm *footprint constante* por construção, enquanto o de WiSARD/ClusWiSARD é **dependente dos dados** — ≈ 2 KB na mediana, mas chegando a ≈ 116 KB nas bases estruturadas (*Weather*, *Modbus*), onde o endereçamento amplo precisa memorizar muitos endereços distintos para alcançar $F1 \approx 0,93$. É essa memorização que paga o ganho de F1.

7.7 Fronteira de Pareto F1 $\times$ custo

A Figura 19 resume o *trade-off* F1 $\times$ custo em dois eixos, com cada ponto sendo um modelo na sua melhor configuração média *per-device*. No eixo de tempo de treino (painel esquerdo), os

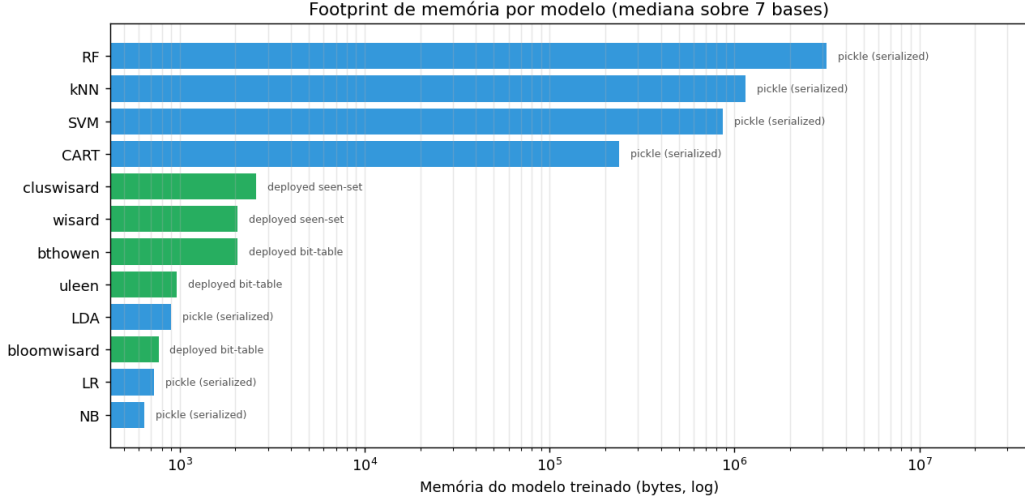


Figura 18: Memória de estado implantado por modelo (`deployedSizeBytes`, mediana sobre 7 bases, escala log). Na mediana a família sem peso ocupa 0,75–2,5 KB, contra 0,2–3 MB dos *baselines* de árvore; o *footprint* de Bloom/BTHOWeN/ULEEN é fixo, o de WiSARD/ClusWiSARD é dependente dos dados (até ≈ 116 KB nas bases estruturadas).

modelos de árvore dominam: RF e CART atingem $F1 \approx 0,83$ em $\approx 0,04$ s e o kNN 0,789 em $\approx 0,004$ s; o trio WiSARD/ClusWiSARD treina em $\approx 0,5$ s com $F1 \approx 0,82$ e o BTHOWeN em 0,09 s com 0,76; LSTM e ULEEN são dominados (tempo alto). É no eixo de memória (painel direito) que a família se distingue — mas como *trade-off*, não como domínio absoluto. As variantes de filtro (Bloom/BTHOWeN/ULEEN) formam a fronteira de baixo custo com *footprint fixo* de $\approx 0,75$ –2 KB a $F1 \approx 0,76$, três ordens de magnitude abaixo das árvores; WiSARD e ClusWiSARD recuperam o melhor F1 da família (0,82) com ≈ 2 KB *medianos*, ainda muito abaixo das árvores, mas com *footprint* dependente dos dados que cresce nas bases onde fazem a memorização. Para *deployment* em *edge* sob restrição *dura* de memória, as variantes de filtro são a escolha; para máximo F1 dentro da família a um custo de memória ainda sub-árvore, WiSARD/ClusWiSARD.

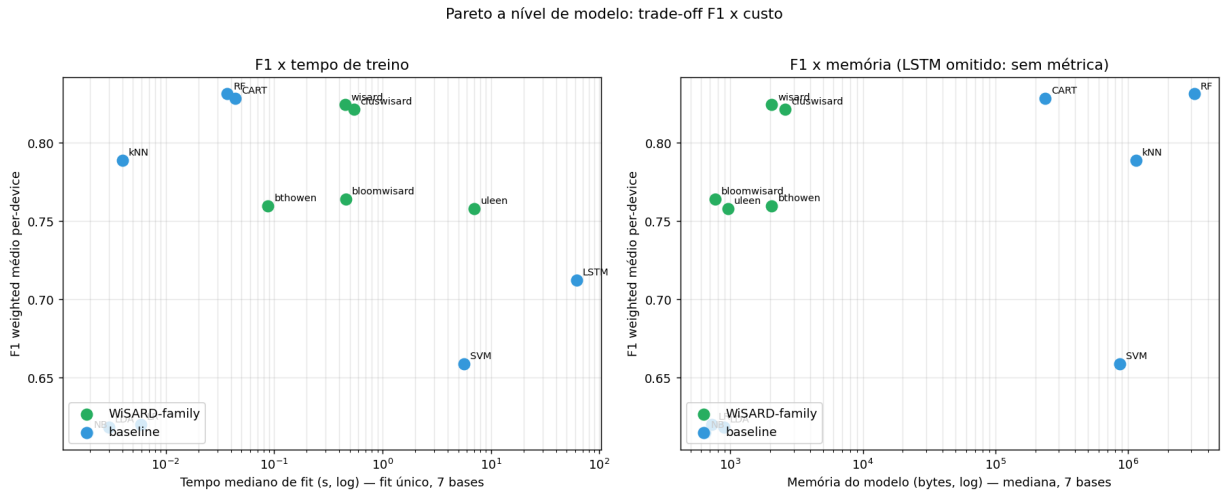


Figura 19: Pareto a nível de modelo (melhor configuração média) em dois eixos de custo: $F1 \times$ tempo de *fit* (esquerda) e $F1 \times$ memória (direita), per-device binário. Os modelos de árvore dominam em tempo; a família WiSARD ocupa KBs contra MBs dos *baselines* em memória, com *footprint* fixo para Bloom/BTHOWeN/ULEEN e dependente dos dados para WiSARD/ClusWiSARD. O LSTM é omitido no painel de memória por não ter métrica comparável.

7.8 Síntese dos resultados

A Tabela 11 consolida o resultado em uma única visão: F1 médio sobre todas as bases per-device, F1 *combined* binário, F1 *combined* multi-classe e tempo mediano de *fit*.

Tabela 11: Síntese final: F1 *weighted* médio (sobre 7 bases $\times$ 4 *folds*) per-device, F1 *combined* binário, F1 *combined* multi-classe e tempo mediano de *fit* único (`bench_cost`, inclui codificação).

Negrito = melhor por coluna.

Modelo	F1 per-dev.	F1 comb. bin.	F1 comb. multi.	Tempo (s, med.)
<i>Baselines</i>				
LR	0,620	0,621	0,296	0,01
LDA	0,618	0,629	0,309	0,00
kNN	0,789	0,820	0,541	0,00
RF	0,831	0,858	0,602	0,04
CART	0,828	0,854	0,598	0,04
NB	0,615	0,601	0,028	0,00
SVM	0,659	0,623	0,294	5,64
LSTM	0,712	0,758	0,449	61,94
<i>WiSARD-family</i>				
WiSARD	0,824	0,724	0,413	0,45
ClusWiSARD	0,822	0,725	0,413	0,55
BloomWiSARD	0,764	0,718	0,405	0,46
BTHOWeN	0,760	0,514	0,241	0,09
ULEEN	0,758	0,745	0,425	7,04

7.9 Leitura dos resultados

Per-device binário. RF/CART dominam ($\approx 0,83$), seguidos por kNN (0,789); a família WiSARD fica logo atrás, com WiSARD (0,824) e ClusWiSARD (0,822) liderando o grupo *empatadas*, a cerca de 1 pp de RF/CART e à *frente* de kNN, LSTM (0,712) e SVM (0,659). Em Modbus, WiSARD/ClusWiSARD (0,95) chegam a *igualar* RF/CART. Em bases com *leak* categórico forte (Fridge, Garage_Door), todos saturam; em bases sem sinal (Motion_Light), todos colapsam para predição majoritária.

Combined binário. RF/CART (com 10 árvores e Gini) lideram ($\approx 0,85$). A família WiSARD cobre 0,51–0,745; o melhor (ULEEN em 0,745) fica 11 pp atrás de RF e é comparável ao LSTM (0,758), com WiSARD, ClusWiSARD e BloomWiSARD agora em $\approx 0,72$ (acima dos 0,66–0,69 da varredura original); o BTHOWeN destoa em 0,514.

Combined multi-classe. O *gap* entre os grupos cresce: RF/CART em $\approx 0,60$; kNN em 0,54; LSTM em 0,45. A família WiSARD fica entre 0,24 (BTHOWeN) e 0,43 (ULEEN), com ULEEN (0,425), WiSARD e ClusWiSARD (0,413) à frente de LR/LDA/SVM ($\approx 0,30$), bem acima de NB (0,03), mas ainda abaixo dos modelos baseados em árvore.

Custo computacional. A vantagem da família WiSARD não está no tempo de CPU, e sim na memória. Em tempo de treino, o BTHOWeN é o mais rápido da família (0,09s mediano por *fit*, cerca de **705 $\times$ mais rápido que o LSTM**), mas RF (0,04s) e kNN (0,004s) treinam em tempo comparável ou menor com F1 superior, de modo que a família não domina o eixo de tempo. O ULEEN, com a configuração *justa* (20 *epochs*, cinco sub-modelos), passa a gastar $\approx 7s$ — mais que o SVM —, mas isso é em parte artefato de sua implementação de referência em *Python*, e não do algoritmo (**ressalva de implementação**: o trio WiSARD/ClusWiSARD/BloomWiSARD roda em C++; BTHOWeN e ULEEN em *Python*, de modo que seu tempo de

CPU superestima o custo de uma implementação otimizada ou em *hardware* dedicado). É na memória que a separação aparece, sob a métrica única de estado implantado: as variantes de filtro (Bloom/BTHOWeN/ULEEN) ocupam $\approx 0,75\text{--}2\text{ KB}$ *fixos*, três ordens abaixo das árvores ($0,2\text{--}3\text{ MB}$); WiSARD/ClusWiSARD recuperam o melhor F1 da família com $\approx 2\text{ KB}$ *medianos* (porém dependentes dos dados). A escolha *dentro* da família é então um *trade-off*: filtro de *footprint* fixo e mínimo (Bloom/BTHOWeN/ULEEN, $F1 \approx 0,76$) versus o F1 mais alto de WiSARD/ClusWiSARD ($0,82$) a um custo de memória ainda sub-árvore.

8 Discussão e limitações

Esta seção discute as limitações da reprodução em ordem de impacto. A divergência de tamanho entre os CSVs públicos e o dataset do paper (§8.1) é a dominante; seguem a escolha de não usar `clean_df` (§8.2), a métrica F1 inferida empiricamente, a semente e os hiperparâmetros não publicados pelo paper, a contagem das “22 *features*”, a substituição do SVM no *combined* e a cobertura reduzida do *sweep* combinado, encerrando com o resumo das limitações.

8.1 Divergência de tamanho é a limitação dominante

O *achado* de §3.6 domina todas as demais limitações da reprodução: os CSVs públicos `Train_Test_IoT_*.csv` são um *subset re-balanceado* do dataset original. A classe `normal` foi sub-amostrada de $\sim 35\,000$ para $\sim 15\,000$ amostras por dispositivo (razão $\approx 0,43$), enquanto a classe `attack` foi preservada na íntegra. Isso significa que mesmo o pipeline mais fiel ao paper não consegue reproduzir os números exatos das Tabelas 10–13, e que qualquer comparação *ours* vs. *paper* deve ser lida como “ordem de grandeza” e não como “match exato”.

A consequência concentra-se na tarefa *combined* multi-classe. Confrontando a Figura 5 do paper com os CSVs públicos, os *ataques* são idênticos linha a linha (`backdoor` 35 k, `ddos` 25 k, `injection` 35 k, `password` 35 k, `ransomware` 16 030, `scanning` 3 973, `xss` 6 116); a *única* diferença é a classe `normal` (245 000 no paper, 105 000 nos CSVs públicos). Como a acurácia e o F1 *weighted* multi-classe seguem a prevalência de `normal` — a Tabela 13 do paper reporta acurácia de LR = 0,61, igual à sua fração de `normal` —, nossos números ficam sistematicamente abaixo: com 40% de `normal` contra os 61% do paper, há menos da classe majoritária “fácil” para sustentar a métrica. A tarefa *combined* binária, ao contrário, reproduz bem (§5): separar ataque de `normal` depende pouco do volume da classe majoritária.

O *Naïve Bayes* é o único modelo cujo *gap* vai além da prevalência. Nossa acurácia multi-classe (0,106) fica *abaixo* do *baseline* de maioria (0,40), enquanto a do paper (0,54) fica próxima da sua prevalência. A causa é uma patologia do `GaussianNB`: as *features* de proveniência (colunas de dispositivo preenchidas com zero) e as variáveis quase-constantes por classe produzem variâncias degeneradas que levam o modelo a prever `ddos` em vez de `normal` (aumentar `var_smoothing` recupera apenas parcialmente, e foge do `GaussianNB` padrão do paper). Acreditamos que os deltas residuais de §5 ($|\Delta| < 0,15$ em 91% dos pares) são amplamente explicados pela divergência de `normal`, e que sob o dataset original do paper a aderência seria substancialmente maior.

8.2 A escolha entre `clean_df` e não-`clean_df`

O paper afirma, em sua §VI-A1, que features categóricas são encodadas como “high→1, low→0”. Isso sugere que o paper operou sobre *features já limpas* (sem variantes de *whitespace*). Nos CSVs públicos, no entanto, as quatro variantes (`'high'`, `'high '`, `'low'`, `'low '`) coexistem; o `LabelEncoder` cru as trata como classes distintas.

Há *duas* opções, e **nenhuma reproduz o paper perfeitamente**:

- **Com `clean_df`**: as variantes são normalizadas; o sinal residual de encoding desaparece; F1 em `Fridge` e `Garage_Door` cai para predição majoritária — nossos números ficam muito *abaixo* do paper.

- **Sem `clean_df`** (escolhido): o encoding cru preserva qualquer sinal residual; o modelo aprende o que provavelmente aprendeu nos dados originais do paper; para algumas bases isso dá *mais* sinal do que o paper teve (variantes *whitespace* são informação extra), e nossos Δ ficam positivos.

Adotamos a segunda opção — a escolha que mais se aproxima de uma reprodução honesta dada a versão de dataset que temos — e documentamos explicitamente que isso pode inflar nossos números em `Fridge` para LR/SVM. O `clean_df` permanece útil em §3 para mostrar a magnitude do *leak* introduzido pelo encoding, mas não é parte do pipeline *paper-faithful*.

8.3 Métrica F1 inferida empiricamente

Como o paper não especifica a chamada exata de `sklearn`, a métrica adotada (**weighted**) é inferência empírica (§4.1.1). Se evidência adicional surgir (por exemplo, código do paper sendo publicado) reservamos o direito de re-rodar com a métrica correta. Nesse cenário, todos os resultados desta versão podem ser recomputados em poucas horas sobre os *JSONLs*, sem re-treinar modelos: `f1` (binário), `f1_macro` e `f1_weighted` são gravadas simultaneamente em cada execução.

8.4 Semente e hiperparâmetros não publicados pelo paper

O paper não publica a semente de aleatoriedade do *split* 80/20, dos *folds* nem da inicialização de RF/SVM/LSTM; fixamos `random_state=42` em todos os nossos experimentos, sem garantia de paridade com a partição original. Além disso, o paper especifica apenas os hiperparâmetros de cabeçalho de cada modelo (RF com 10 árvores e critério Gini, kNN com $k = 5$, SVM RBF com `gamma='auto'`) e deixa os demais (`C`, `solver`, `max_iter`, `class_weight`) nos *defaults* do `scikit-learn` 0.22.1 da época. Como esses *defaults* mudaram entre versões, o uso do `scikit-learn` atual é uma fonte adicional e inevitável de divergência.

8.5 Combined: as “22 features” do paper são contagem de colunas

O paper §VI-C reporta que o *combined* foi montado “with a total of 22 features”. Nosso *combined* (via `build_combined_dataset`) tem 17 *features* de sensor — o UNION das colunas das sete bases. A diferença é *contável*, não conceitual: as 22 são a contagem de colunas do CSV combinado, isto é, as 17 *features* de sensor mais `date`, `time`, `timestamp`, `label` e `type`. O próprio paper, em sua §VI-A1, declara descartar `date`, `time` e `timestamp` do vetor de *features* (“were omitted... as they may cause some ML methods to overfit”), exatamente como fazemos via `drop_temporal_leak`. Logo, **ambos os pipelines treinam sobre o mesmo conjunto de ~17 features de sensor** — não há discrepância de *features* de modelagem, e em particular o paper *não* obteve sinal extra por reter colunas temporais.

Em nossa construção do *combined*, as numéricas faltantes do *union* das sete bases são preenchidas com zero (verificamos que trocar a imputação por mediana não altera os F1 dos *baselines*). As *features* categóricas, ao terem NaN convertido em um *label* próprio pelo `LabelEncoder`, viram um proxy de “dispositivo de origem”: apenas `Fridge` preenche `temp_condition`, apenas `Garage_Door` preenche `sphone_signal`, e assim por diante. *Esse leak* de proveniência não estava no protocolo do paper e é uma fonte adicional de divergência, especialmente para os modelos mais ávidos por encoding categórico (LR, kNN, NB).

8.6 Substituição do SVM RBF no *combined*

O SVM com kernel RBF é $\mathcal{O}(n^2)$ em tempo de treinamento, inviável em ~210 000 linhas do *combined*. Como o SVM é um dos oito *baselines* do paper e queremos preservá-lo na comparação, substituímos apenas o *kernel* RBF por `LinearSVC` para $n > 50\text{k}$, registrando o substituto na coluna `effective_model` do *JSONL*.

8.7 Cobertura limitada do *sweep combined*

Os experimentos *per-device* cobrem todo o produto cartesiano de hiperparâmetros de cada modelo. Os experimentos *combined* usam apenas uma configuração canônica por modelo (geralmente a melhor configuração descoberta no *per-device*), por motivo de custo computacional. Isso significa que os F1 *combined* reportados são limites inferiores: um *sweep* completo poderia encontrar configurações ligeiramente melhores.

8.8 Generalização

Todos os resultados acima são específicos aos CSVs públicos atuais do TON_IoT. Reprodutibilidade exata requer a versão original do dataset (não publicamente disponível) ou que os autores publiquem o *seed* de re-amostragem que produziu os CSVs públicos. Generalização para outros datasets IoT requer revalidar pelo menos o pipeline de auditoria (Cramér’s V, leak temporal) antes de declarar qualquer ranking entre modelos.

8.9 Resumo das limitações

- Versão de dataset diferente do paper (classe `normal` sub-amostrada para 43% do volume original).
- Métrica F1 inferida empiricamente (não declarada no paper).
- Semente de aleatoriedade não publicada pelo paper; fixamos `random_state=42`, sem paridade garantida com a partição original.
- Hiperparâmetros não-cabeçalho dos *baselines* deixados nos *defaults* do `scikit-learn`, que diferem entre versões.
- Pipeline *paper-faithful* sem `clean_df` — preserva *leaks* do encoding (decisão honesta, mas documentada).
- Leak de proveniência no *combined* (NaN categórico vira proxy de dispositivo de origem).
- SVM RBF substituído por `LinearSVC` no *combined*.
- Cobertura reduzida do *sweep combined*.

9 Conclusão

Este relatório consolidou o Trabalho 1 da disciplina de Redes Neurais Sem Peso (mestrado, 2026/1), em que reproduzimos fielmente os oito *baselines* de Alsaedi et al. (2020) [4] no dataset TON_IoT e avaliamos cinco modelos da família WiSARD sob *exatamente o mesmo protocolo*.

Quanto à reprodução do paper. Sob o protocolo *paper-faithful* (*split* 80/20, $k = 4$ *StratifiedKFold*, métrica F1 `weighted`), nossa reprodução dos oito *baselines* bate com as Tabelas 10–11 do paper dentro de $|\Delta| < 0,15$ em 91% dos pares (modelo, base), com média de $\Delta = -0,009$. Os deltas residuais concentram-se nas bases mais afetadas pela divergência de tamanho entre os CSVs públicos e o dataset original do paper (sub-amostragem da classe normal para 43% do volume original), o que documentamos como o fator limitante dominante da reprodução.

Quanto à família WiSARD. Sob o mesmo protocolo, a família WiSARD apresenta desempenho competitivo na maioria das bases *per-device*: WiSARD (0,824) e ClusWiSARD (0,822) lideram o grupo *praticamente empatadas*, a cerca de um ponto percentual de Random Forest e CART ($\approx 0,83$) e à frente de kNN (0,789), LSTM (0,712) e SVM (0,659); BloomWiSARD (0,764), BTHOWeN (0,760) e ULEEN (0,758) vêm logo atrás. Essa paridade WiSARD/ClusWiSARD só

emergiu após unificarmos os tetos de capacidade do *sweep* (§6.8): com tetos desiguais, ClusWiSARD aparecia 0,04 atrás por artefato de cobertura, não de modelo. Em bases com *leak* categórico forte (Fridge, Garage_Door), toda a família satura em $F1 \approx 1,000$; em bases sem sinal discriminativo (Motion_Light), todos colapsam para predição majoritária. Em Modbus, com estrutura não-linear nos códigos FC, WiSARD e ClusWiSARD com endereçamento amplo chegam a 0,95, *igualando* os modelos de árvore (RF/CART $\approx 0,94$). No *combined* multi-classe, o *gap* cresce: a família fica em 0,24–0,43, com ULEEN (0,425), WiSARD e ClusWiSARD (0,413) à frente de LR/LDA/SVM ($\approx 0,30$) e bem acima de NB (0,03), mas distante de RF/CART (0,60).

Quanto à viabilidade em *edge*. A discussão mais relevante deste trabalho não está nos números brutos de F1, mas no *trade-off* $F1 \times$ custo, e nesse *trade-off* o eixo decisivo é a memória. O BTHOWeN treina em 0,09s medianos (cerca de **705× mais rápido que o LSTM**) e entrega $F1 \approx 0,76$. Em $F1 \times$ tempo de CPU, porém, os modelos de árvore ainda dominam: RF e kNN treinam tão rápido quanto o BTHOWeN e com $F1$ superior. A vantagem genuína da família aparece na memória, sob uma métrica única de estado implantado: as variantes de filtro (BloomWiSARD, BTHOWeN, ULEEN) são implantáveis em 0,75–2 KB *fixos*, cerca de três ordens de magnitude abaixo dos modelos de árvore (RF ≈ 3 MB, kNN ≈ 1 MB), a um custo de poucos pontos de $F1$; WiSARD e ClusWiSARD recuperam o melhor $F1$ da família (0,82) com ≈ 2 KB *medianos* — ainda muito abaixo das árvores, mas com *footprint* dependente dos dados que cresce nas bases estruturadas. É esse *footprint* reduzido — e a inferência em *hardware* dedicado documentada nos artigos originais de BTHOWeN e ULEEN, que um *benchmark* em CPU não captura — que torna a família preferível para *deployment* em hardware embarcado com restrição de memória.

Trabalhos futuros. As direções naturais para este trabalho são (i) acesso ao dataset original do paper, eliminando a divergência de tamanho que limita a reprodução; (ii) *dispatch* por *feature*, escolhendo o melhor termômetro para cada variável em vez de aplicar um único termômetro uniformemente, uma vez que a alocação **non-uniform** já entrou no *sweep* atual; (iii) avaliação do BTHOWeN em *ULEEN-style training* (*epochs+learning rate*) para verificar se o tempo extra de treinamento compensa pelo $F1$ ganho; (iv) extensão dos *sweeps* para hardware-target (latência de inferência e *memory footprint* sob restrição de FPGA), seguindo o protocolo do BTHOWeN original.

Conclusão final. A família WiSARD é uma família viável de modelos sem peso para detecção de anomalia em IoT, com perfil $F1 \times$ custo claramente distinto e em geral favorável frente aos *baselines* clássicos para o cenário *edge*. Os resultados não substituem os modelos baseados em árvore quando $F1$ puro é o critério único, mas oferecem redução de cerca de três ordens de magnitude em *footprint* de memória (variantes de filtro, com *footprint* fixo) a uma perda relativamente pequena em $F1$, nas bases sem *leak* forte. Duas contribuições metodológicas acompanham esse resultado: a unificação dos tetos de capacidade do *sweep*, que tornou a comparação intra-família justa (revelando que a aparente liderança da WiSARD sobre a ClusWiSARD era artefato de cobertura de *grid*), e a métrica única de estado implantado que coloca todos os modelos sem peso no mesmo eixo de memória. A maior contribuição, porém, é a explicitação dos três problemas estruturais dos CSVs públicos do TON_IoT (*leak* categórico, *leak* temporal universal, divergência de tamanho contra o paper original) — o que esperamos sirva de referência para trabalhos futuros sobre esse dataset.

Disponibilidade de código e dados. Todo o pipeline — *notebook* de referência (06_consolidado_grupos.ipynb), *scripts* de *sweep* e *benchmark*, e os JSONL/JSON de resultados que sustentam cada número reportado — está disponível em <https://github.com/muanlartins/masters/tree/toniot-wisard-repro>. A biblioteca `wisardpkg` (com o termômetro logarítmico e a métrica `deployedSizeBytes`) é fixada por *commit* em `requirements.txt`. O dataset TON_IoT Train_Test_IoT_*.csv é público (UNSW Canberra Cyber).

Uso de assistentes de IA. O texto deste relatório foi redigido com assistência de IA generativa (Claude Code), a partir dos números, figuras e estrutura definidos pelos autores no *notebook* de consolidação. Toda a metodologia, todos os experimentos e todas as conclusões são de autoria humana.

Referências

- [1] Igor Aleksander, Massimo De Gregorio, Felipe M. G. França, Priscila M. V. Lima, and Helen Morton. A brief introduction to Weightless Neural Systems. *Proceedings of the European Symposium on Artificial Neural Networks (ESANN)*, pages 299–305, 2009.
- [2] Igor Aleksander, W. V. Thomas, and P. A. Bowden. WISARD: A radical step forward in image recognition. *Sensor Review*, 4(3):120–124, 1984.
- [3] Yazeed Alotaibi and Mohammad Ilyas. Ensemble-learning framework for intrusion detection to enhance internet of things’ devices security. *Sensors*, 23(12):5568, 2023.
- [4] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access*, 8:165130–165150, 2020.
- [5] Hafsa Benaddi, Mohammed Jouhari, Nouha Laamech, Anas Motii, and Khalil Ibrahimi. Lightweight intrusion detection in IoT via SHAP-guided feature pruning and knowledge-distilled Kronecker networks, 2025.
- [6] Tim M. Booij, Irina Chiscop, Erik Meeuwissen, Nour Moustafa, and Frank T. H. den Hartog. ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets. *IEEE Internet of Things Journal*, 9(1):485–496, 2022.
- [7] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [8] D. O. Cardoso, D. S. Carvalho, D. S. F. Alves, D. F. P. de Souza, H. C. C. Carneiro, C. E. Pedreira, P. M. V. Lima, and F. M. G. França. Financial credit analysis via a clustering weightless neural classifier. *Neurocomputing*, 183:70–78, 2016.
- [9] T. Cover and P. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.
- [10] N. Dharini, V. S. Janani, and J. Katiravan. Efficient detection of intrusions in ToN-IoT dataset using hybrid feature selection approach. *Scientific Reports*, 16:7763, 2026.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [12] IAZero Lab, UFRJ. wisardpkg: A weightless neural network library. <https://github.com/IAZero/wisardpkg>, 2020. Accessed 2026.
- [13] Nour Moustafa. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society*, 72:102994, 2021.
- [14] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [15] Anton Raskovalov, Nikita Gabdullin, and Vasily Dolmatov. Investigation and rectification of NIDS datasets and standardized feature set derivation for network attack detection with graph neural networks, 2022.
- [16] Leandro Santiago, Lucca Verona, Fabio Rangel, Fabricio Firmino, Priscila M. V. Lima, and Felipe M. G. França. Memory efficient weightless neural network using Bloom filter. In *Proceedings of the European Symposium on Artificial Neural Networks (ESANN)*, pages 419–424, 2020.

- [17] Aigul Shaikhanova, Oleksandr Kuznetsov, Aigerim Tokkuliyeva, Kalybek Ayapbergenov, Sailaubek Olzhas, and Tolep Danir. Security audit of IoT device networks: A reproducible machine learning framework for threat detection and performance benchmarking. *Sensors*, 25(24):7519, 2025.
- [18] Yousef Sharrab, Abdel-Rahman Al-Ghuwairi, Alaa Alomoush, Ahmad Al-Husini, Dema Al-Burgan, and Izzat Alsmadi. Analysis and evaluation of ToN IoT windows and garage door datasets. In *2025 5th Intelligent Cybersecurity Conference (ICSC)*, pages 214–218, 2025.
- [19] Zachary Susskind, Aman Arora, Igor D. S. Miranda, Aluisio Stefano O. Bacellar, Luis A. Q. Villon, Rafael F. Katopodis, Leandro S. de Araujo, Diego L. C. Dutra, Priscila M. V. Lima, Felipe M. G. França, Mauricio Breternitz Jr., and Lizy K. John. ULEEN: A novel architecture for ultra-low-energy edge neural networks. *ACM Transactions on Architecture and Code Optimization*, 20(4):1–24, 2023.
- [20] Zachary Susskind, Aman Arora, Igor D. S. Miranda, Luis A. Q. Villon, Rafael F. Katopodis, Leandro S. de Araujo, Diego L. C. Dutra, Priscila M. V. Lima, Felipe M. G. França, Mauricio Breternitz Jr., and Lizy K. John. BTHOWeN: A hardware-friendly weightless neural network with hashed lookup tables. In *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*, pages 1–6, 2022.